

UNIVERSIDADE FEDERAL DE MINAS GERAIS
Escola de Engenharia
Programa de Pós-Graduação em Engenharia de Produção

Bernardo Bastos Pereira Moreira

A new combinatorial approach to solve the Job Sequencing and Tool
Switching Problem

Belo Horizonte
2026

Bernardo Bastos Pereira Moreira

**A new combinatorial approach to solve the Job Sequencing and Tool
Switching Problem**

Final Version

Thesis proposal presented to the Graduate Program in Production Engineering of the Federal University of Minas Gerais in partial fulfillment of the requirements for the degree of Master in Production Engineering.

Advisor: Mauricio de Souza
Co-Advisor: Annegret Wagler, Rafael Colares and Ricardo de Camargo

Belo Horizonte
2026

[Ficha Catalográfica em formato PDF]

A ficha catalográfica será fornecida pela biblioteca. Ela deve estar em formato PDF e deve ser nomeada como “**fichacatalografica.pdf**” e inserida na raiz do projeto, junto com o arquivo **main.tex**.

[Folha de Aprovação em formato PDF]

A folha de aprovação deve estar em formato PDF e deve ser nomeada como “**folhadeaprovacao.pdf**” e inserida na raíz do projeto, junto com o arquivo **main.tex**.

Dedicuum cest laborae a quelquis personatum que ajudorat a facirelo.

Acknowledgments

To do.

“*Truth and lie are opposite things.*”
(Unkown)

Resumo

[TO BE UPDATED]

Este documento descreve uma dissertação de mestrado utilizando uma proposta de nova formulação exata para solucionar o Job Sequencing and Tool Switching Problem (SSP). O SSP é o problema mais comum oriundo do planejamento da produção em sistemas flexíveis de manufatura, no qual uma máquina tem a capacidade de conter múltiplas ferramentas de uma só vez e as tarefas a serem processadas requerem configurações específicas de ferramentas.

Inicialmente, nós exploramos a literatura do SSP, suas principais características e variações, diferentes abordagens para resolvê-lo e alguns exemplos de aplicações práticas reais nas quais esse problema aparece. Em seguida, nós explicamos as duas melhores formulações exatas que conhecemos para resolver o SSP. Após disso, nós utilizamos uma nova proposta de formulação para resolver o SSP baseado no Problema do Caixeiro Viajante (TSP). Resultados preliminares de relaxações lineares de todos os modelos para instâncias pequenas são apresentados para justificar a necessidade de maior exploração do potencial da nova formulação. Finalmente, são apresentadas algumas ideias de métodos heurísticos para geração de configurações de ferramentas a serem usadas como parâmetros na nova formulação.

Os resultados obtidos por esse trabalho de qualificação mostram resultados promissores em relação à qualidade do *lower bound* obtido usando a nova formulação em comparação com as melhores formulações da literatura.

Palavras-chave: Otimização Combinatória, Job Sequencing and Tool Switching Problem, Traveling Salesman Problem

Abstract

[TO BE UPDATED]

This work describes the qualification work regarding the proposition of a new exact formulation to solve the Job Sequencing and Tool Switching Problem (SSP). The SSP is the main and most simple form of problem that arises in the context of flexible manufacturing systems planning, in which a machine can hold many tools at once and the jobs to be processed have unique required tools configuration.

First, we explore the literature of the SSP, its main features and variations, different approaches to solve it and some real world examples in which this problem is presented. Following that, we explain the two best exact formulations known to us to solve the SSP. We then propose a new formulation to solve the SSP based on the Travelling Salesperson Problem (TSP). Preliminary results of linear relaxations from all models to small instances are presented in order to justify the need of more exploration of the new model. Finally, some ideas for heuristic methods to generate input configurations for the new model are presented.

The results obtained by this qualification work show promising results regarding the quality of the lower bound solutions obtained using the new formulation in comparison with the best formulations in the literature.

Keywords: Combinatorial Optimization, Job Sequencing and Tool Switching Problem, Traveling Salesman Problem

List of Figures

- 1.1 Graph from [9] 22
- 3.1 Computational time (s) to solve new formulation for 10 small instances by number of extra configurations provided. 32
- 3.2 Flow diagram describing the extra configuration creation process of a multi-start run in the heuristic method. 55
- 4.1 Distribution of instances for best to worst solution gap by standard deviation of obtained solutions. 59
- 4.2 Percentage of multi-start runs that found the known optimal solution for each instance. 60
- 4.3 Distribution of instances for Average Improvement by applying KTNS by Largest Improvement by applying KTNS, both in number of switches reduced. 61
- 4.4 Distribution of instances for best obtained upper bound before applying KTNS and worst obtained upper bound before applying KTNS for each multi-start run, both in number of switches, separating for Random Factor used in the run. 71
- 4.5 Box plot of the obtained upper bound values before applying the KTNS policy for each subgroup of instances of datB and each value of Random Factor. . . . 72
- 4.6 Box plot of the obtained upper bound values before applying the KTNS policy for each subgroup of instances of datC and each value of Random Factor. . . . 73
- 4.7 Box plot of the obtained upper bound values before applying the KTNS policy for each subgroup of instances of datD and each value of Random Factor. . . . 74
- 4.8 Distribution of instances for best obtained upper bound before applying KTNS and worst obtained upper bound before applying KTNS for each multi-start run, both in number of switches, separating for Next Tool Strategy used in the run. 75
- 4.9 Box plot of the obtained upper bound values before applying the KTNS policy for each subgroup of instances of datB and each option of Next Tool Strategy. 76
- 4.10 Box plot of the obtained upper bound values before applying the KTNS policy for each subgroup of instances of datC and each option of Next Tool Strategy. 77
- 4.11 Box plot of the obtained upper bound values before applying the KTNS policy for each subgroup of instances of datD and each option of Next Tool Strategy. 78

4.12 Distribution of instances for best obtained upper bound before applying KTNS
and worst obtained upper bound before applying KTNS for each multi-start
run, both in number of switches, separating for Next Job Strategy used in the
run. 79

4.13 Box plot of the obtained upper bound values before applying the KTNS policy
for each subgroup of instances of datB and each option of Next Job Strategy. . 80

4.14 Box plot of the obtained upper bound values before applying the KTNS policy
for each subgroup of instances of datC and each option of Next Job Strategy. . 81

4.15 Box plot of the obtained upper bound values before applying the KTNS policy
for each subgroup of instances of datD and each option of Next Job Strategy. . 82

List of Tables

1	Optimal value, complete MIP models runtime (s) and linear relaxation values for the three formulations for small instances (datA group).	30
2	Value of Random Factor and Next Job and Next Tool strategies elected for each of the 12 multi-start runs that will be performed for each instance.	38
3	Optimal solution, obtained upper bound with heuristic and computational times for heuristic and SSPMF and CGL models for datB instances for which optimal was found with exact models.	58
4	Best upper bound obtained by a multi-start, upper bound obtained before and after the KTNS policy for the mixing and improving phase and improvement from that phase (in switches) for instances in which the mixing and improving phase was able to improve the solution.	63
5	Obtained upper bounds with ILS, HGS and New Heuristics and time in seconds to obtain the upper bound for every instance from Catanzaro dataset [4].	65
6	Obtained upper bounds with ILS, HGS and New Heuristics and time in seconds to obtain the upper bound for every instance from Crama dataset [8]	66
7	Obtained upper bounds with ILS, HGS and New Heuristics and time in seconds to obtain the upper bound for GroupA from Yanasse dataset	67
8	Obtained upper bounds with ILS, HGS and New Heuristics and time in seconds to obtain the upper bound for GroupB from Yanasse dataset	67
9	Obtained upper bounds with ILS, HGS and New Heuristics and time in seconds to obtain the upper bound for GroupC from Yanasse dataset	68
10	Obtained upper bounds with ILS, HGS and New Heuristics and time in seconds to obtain the upper bound for GroupD from Yanasse dataset	68
11	Obtained upper bounds with ILS, HGS and New Heuristics and time in seconds to obtain the upper bound for GroupE from Yanasse dataset	68

Contents

Introduction

14

1

Related Works

18

1.1

Early Works

18

1.2

The CGL formulation

19

1.3

The SSPMF formulation

21

1.4

Heuristics and Metaheuristics

24

2

Proposed model and Preliminary Results

26

2.1

A non-compact formulation for the Tool Switching Problem

27

2.2

Proposed work

28

3

Heuristics based on the proposed model for larger instances

31

3.1

Motivation for configuration generation heuristic methods

31

3.2

General guidelines to find good configuration sets

32

3.3

Summary of the heuristic method

34

3.4

Step 1: Defining parameters and strategies for the multi-start iterations . .

37

3.4.1

Next Job Strategies

38

3.4.1.1

Next Job Strategy: Random

38

3.4.1.2

Next Job Strategy: Least Covered Job

39

3.4.1.3

Next Job Strategy: Most Available Configurations

40

3.4.1.4

Next Job Strategy: Lowest Share of Satisfying Configurations

41

3.4.2

Next Tool Strategies

41

3.4.2.1

Next Tool Strategy: Random

42

3.4.2.2

Next Tool Strategy: Most Frequent Tool

43

3.4.2.3

Next Tool Strategy: Most Joint-Frequent Tool

43

3.4.3

Maximum Amount of Configurations

44

3.4.4

Maximum Amount of Multi-Start Iterations

44

3.5

Step 2: Multi-Start Configuration Generation Phase

45

3.5.1

Generation of base configurations

45

3.5.1.1

Multi-job Configuration Creator

46

3.5.2

Generation of extra configurations

46

3.5.2.1

Single-job Configuration Creator

48

3.5.2.2	High-level Configuration Creation	48
3.5.3	Optimization	50
3.5.4	KTNS improvement	50
3.6	Step 3: Mixing and Improving Configurations Phase	51
3.6.1	Selection of configurations from upper bound solutions in multi-start phase	51
3.6.2	Optimization and KTNS improvement	53
4	Results and discussion	56
4.1	General results of the heuristic method	57
4.1.1	General results for medium-sized instances with known optimal solution	57
4.1.2	General results of the heuristic method for all instances from [4]	58
4.1.2.1	Multi-start results before applying KTNS	60
4.1.2.2	KTNS improvement results	61
4.1.3	Mixing and Improvement phase results	62
4.2	Comparison with the heuristics from [15] and [14] for all instances	64
4.2.1	Results for Catanzaro's instances from [4]	64
4.2.2	Comparison with [14] for Crama's instances from [8]	65
4.2.3	Comparison with [14] for Yanasse's instances from [21]	66
4.3	Comparison between multi-start strategies	69
4.3.1	Random Factor results	70
4.3.2	Next Tool Strategies results	74
4.3.3	Next Job Strategies results	78
5	Conclusions	83
	Conclusion	83
6	Future Work	84
	Conclusion	84
6.1	An idea for a future work: a Clustering method	84
6.1.1	Clustering phase	84
6.1.2	Exploitation phase	85
6.1.3	Linking phase	86
6.2	Similarity Metrics Between Jobs	86
	References	88

Introduction

In the context of rapid market demands transformations and the competitive advantage in reducing costs in production systems, Flexible Manufacturing Systems are highly valuable and can be seen in many applications. One of its main benefits is the ability to process jobs that require different tools much faster and thus cheaper than using more than one machine. In this sense, machine flexibility and material handling flexibility are two main forms in which a manufacturing system can be flexible [8]. The Job Sequencing and Tool Switching Problem (SSP) arises in the context of machine flexibility, as this feature relates to the ease with which tools can be interchanged for use in the production sequence.

In this Flexible Manufacturing Systems, companies try to keep their machines operating for the largest share of the available time, as it lead to a more profitable operation. Many of these machines have the capacity to hold many different tools at once, which leads to the possibility of having a large tool set and thus expanding the companies' capacity to diversify their products, as different final products will require different tools in order to be processed. Considering the scenario of tool switching, the SSP is the problem of sequencing a set of given jobs, each job having its own set of required tools, and minimizing the total number of tool switches required in order to process the jobs in the defined sequence.

The main difference between conventional one-machine sequencing problems and the SSP is the presence of a bounded capacity in the machine regarding the number of tools it can hold at once. The complexity of this problem then comes from the fact that there are more tools than the machine is able to carry at once. This is called tool management and formulates the problem as allocating tools to the machine and simultaneously sequencing the jobs to be processed while optimizing some measure of production performance [8].

There are many practical applications of the SSP. [18] have used an SSP-based approach to solve a warehouse picking problem, in which they had a crane, modeled as the machine, that had to pick different SKUs, modeled as the tools. The crane was unable to pick all SKUs at once, and different orders required different sets of SKUs.

Printed Circuit Boards (PCBs) are a classical example of SSP and SSP variations applications, as the companies are pressured to provide a wide product variety. In this

field, there is also intense competition in the market, which leads to a greater importance of operating the machines at extreme efficiency. Each PCB is a job to be processed that requires a specific set of tools that shall be used by one or more limited-capacity machines. [20].

Another explored practical application of the SSP is in computer systems, as described by [13]. Computers have a limited-capacity fast memory, which could be modeled as the machine, and a slow memory, which can be modeled as the storage of the tools that are not being used. Those memories are used to store pages that are requested, and only pages in the fast memory can be accessed, creating the need to move a requested page to the fast memory and switching it for another page.

There are many versions of sequencing models for flexible systems in regard of what costs are considered in the objective function of the models. For some contexts, the switching cost may be tool dependent, as different tools might require different kinds of setup efforts, impacting time and/or human knowledge in order to perform the change. In other situations, it might be considered that the number of setups is the relevant variable, instead of the number of changes - if, for example, it took the same time to change one tool or many tools, as long as it is done at once. According to [4], the main and most simple version of the switching cost is the uniform and tool independent.

The SSP is commonly divided into two inter-related subproblems:

1. **The Sequencing problem (SP):** aims to determine the sequence in which the jobs should be processed;
2. **The Tooling problem (TP):** aims to minimize the number of tool switches for an already defined sequence of jobs to be processed, respecting for each job in the sequence its needed tools.

[19] showed that the TP is solved optimally in polynomial time by a greedy algorithm named Keep Tool Needed Sooner (KTNS), that can be applied to any given sequence of jobs with its specific required tools. Given this sequence, the KTNS applies a policy in which no tool is selected to be inserted in the machine unless the very next job requires it. When the next job needs a tool that is not currently in the magazine, the tool selected to be kept in the magazine is the one needed sooner in the sequence, thus the name of the policy. Nevertheless, even with a polynomial algorithm to solve the TP, the whole SSP problem, when the SP is not already solved, is NP-hard.

The Tool Switching (TS) problem can be described as follows. We are given a set of jobs $J, |J| = n$ to be scheduled within a fixed timeframe (say, a working shift). A set of tools $T, |T| = m$ is available; and each job $j_i \in J$ requires a subset of tools $T_i \subseteq T$ in order to be fulfilled. In order to process the jobs, a tool repository of capacity b is at disposal. The tool repository can therefore host any subset of tools $C \subseteq T$ as long as $|C| \leq b$. The subset of tools C inside the tool repository is hereafter called a configuration. Without loss of generalization, we can consider that the tool repository is always full, i.e., $|C| = b$. A job $j_i \in J$ can only be processed if all the required tools appear in the tool repository, that is, if $T_i \subseteq C$. We also assume that the tool repository has enough capacity to process all jobs, since otherwise the problem is clearly unfeasible. In other words, for any job $j_i \in J$, we have $|T_i| \leq b$. It is important to note that part of the literature on the topic uses C as the symbol for the magazine capacity. For this work, we will use b as the symbol for the magazine capacity and C for a configuration of tools.

If C does not contain all the required tools to process a job $j_i \in J$, then tool switching operations are needed. A tool switching operation consists of replacing an unnecessary tool $t_j \in C \setminus T_i$ in the repository by a required tool $t_i \in T_i \setminus C$. In order to process j_i , this operation needs to be done for all tools in $T_i \setminus C$. The goal is to schedule all jobs in a way that the number of tool switching operations is minimized. This corresponds to finding an ordering of the jobs that requires the minimum total the number of tool switching operations.

In this work, we will tackle the most common version of the SSP, the Uniform SSP (single-machine, job-independent setup times, minimization of total number of tool changes), but there are many SSP variations worth mentioning:

- Multiple machines: Two or more machines that can either be identical and parallel or different and sequential;
- Sequence-dependent set-up times;
- Multi-objective: includes for example the minimization of both number of tool switches and switching instants or the rotation speed of the tool turret;
- Tool size and capacity constraint variations;
- Tool wear variations: can assume either deterministic or stochastic tool lifetimes.

[3] explores in details a comprehensive literature review on this topic.

To address the Uniform SSP, we will propose a new exact approach proposed by [7] to solve the SSP optimally and compare the results to some of the best formulations proposed so far to solve the SSP. This new formulation is inspired by the Traveling Salesman Problem (TSP) and models each possible tool configuration in the magazine as a node that can be visited in a optimal route. The amount of tool switches needed in order to move from one configuration to another represents the distance between the nodes. The aim would then be to find the optimal path through the nodes such that for each job of the instance we visit at least one configuration that contains its required tool set, minimizing the route length.

Chapter 1

Related Works

As the SSP is known to be a NP-hard problem, there are not many works proposing exact models to solve the SSP optimally. Instead, a large portion of the literature is focused on heuristic methods to solve the SSP [9]. Two main contributions to the exact methods literature for the SSP have been recently made by [4] and [9]. To the best of our knowledge, those are the best two exact formulations for this problem, according to the results published in [9].

1.1 Early Works

[19] are widely recognized for having proposed one of the first exact formulations to solve the SSP. Their model was a position-based formulation, in which binary variables assigned a job to a position in the sequence. They also created binary variables to assign a tool to a position in the sequence and binary variables to indicate a tool switch, equal to one if a tool is present in a position k of the sequence and absent in the position $k + 1$. Their experiments showed that the formulation did not have a good performance. In addition to stating the KTNS policy, they also stated the tool set dominance rule to make some instances smaller: if two jobs i and j are such that $T_i \subseteq T_j$, then job i can be eliminated since it is never suboptimal to execute it immediately after j without any tool change between those two jobs. In this work, we will always work with instances of the SSP containing only jobs requiring non-dominated tool sets.

[12] also had a strong contribution to solving the SSP optimally. First, they also proposed a TSP-based formulation, but their idea was to sequence the jobs themselves, adding a dummy node to the graph and creating binary variables to indicate whether a job was immediately followed by another job in the sequencing part of the SSP. They also used a binary variable to determine whether a tool is inserted into the magazine at the start of each job position. They added the Dantzig, Fulkerson and Johnson [10] subtour

elimination constraints. That formulation had overall better performance results than the formulation from [19], specially because of their linear relaxation value. [12] showed that their formulation could have positive values in the objective function of a linear relaxation, whereas the one from [19] was always equal to zero. After that, they proposed two enumeration algorithms based on their SSP-inspired formulation, one branch-and-cut and one branch-and-bound, the latter having the best results of their work.

1.2 The CGL formulation

[4] proposed a formulation for solving the SSP based on the TSP. In their paper, they compare their results with [19], named Formulation 1 and with [12], named Formulation 2. They then proposed three new formulations that are better versions of the previous ones. Here, we explain, explore and compare their formulation that presented the best results for lower bounds for the problem presented in their article (named Formulation 5). To formulate the SSP based on the TSP, they consider a graph in which the vertices represent the jobs to be sequenced, and the variables decide which jobs are immediately followed by each other.

The formulation then uses a set of jobs J and a set of jobs J_0 that includes all of the jobs in J and also a dummy job 0. They also use the set of tools T and the set of tools required for a specific job, T_j . Binary variables x_{ij} are used to determine if a job j immediately follows job i in the chosen sequence of jobs to be processed.

They also use a concept called a 0-block, which is, for a given tool t , a sequence of jobs starting at a job i and ending at a job j and such that no job in between i and j (but i and j themselves) needs tool t . Using the concept of the 0-block, they introduce a binary decision variable $f_{ij,kl}^t$ that is equal to 1 if vertex l immediately follows vertex k in a subgraph $G_{ij}^t = (V_{ij}^t, A_{ij}^t)$, in which: V_{ij}^t is the set of vertices that includes i, j and all of the jobs from cJ_t for all distinct $i, j \in J$ and such that $t \in T_i \cap T_j$. And A_{ij}^t is the set of subset of arcs that have endpoints in V_{ij}^t , except for (i, j) , arcs coming into i and arcs diverging from j . This means that a tool t is kept in the machine from a job i that requires t , to a job j that also requires t , going through a 0-block of jobs that do not require t .

The CGL formulation is defined as follows:

1.2. The CGL formulation

20

$$\min \sum_{j \in J} |T_j| x_{0j} + \sum_{i,j \in J: i \neq j} \sum_{t \in T_j \cap T_i} (x_{ij} - \sum_{k \in J: k \neq j} \sum_{l \neq i,j} f_{kj,ij}^t) \quad (1.1)$$

$$\text{s.t. :} \quad (1.2)$$

$$\sum_{j \in J_0^i} x_{ij} = 1 \quad \forall i \in J_0 \quad (1.3)$$

$$\sum_{i \in J_0^j} x_{ij} = 1 \quad \forall j \in J_0 \quad (1.4)$$

$$\sum_{i,j \in S: i \neq j} x_{ij} \leq |S| - 1 \quad \forall S \subseteq J_0, 2 \leq |S| \leq n - 1 \quad (1.5)$$

$$\sum_{l \in V_{ij}^t: l \neq k} f_{ij,kl}^t - \sum_{l \in V_{ij}^t: l \neq k} f_{ij,lk}^t = 0 \quad \forall i, j \in J: i \neq j, \forall t \in T_i \cap T_j, \forall k \in^c J_t \quad (1.6)$$

$$\sum_{i \in J_t} f_{il,kl}^t \leq x_{kl} \quad \forall t \in T, \forall k \in^c J_t, \forall l \in J_t \quad (1.7)$$

$$\sum_{j \in J_t} f_{kj,kl}^t \leq x_{kl} \quad \forall t \in T, \forall k \in J_t, \forall l \in^c J_t \quad (1.8)$$

$$\sum_{i,j \in J_t: i \neq j} f_{ij,kl}^t \leq x_{kl} \quad \forall t \in T, \forall k, l \in^c J_t: k \neq l \quad (1.9)$$

$$\begin{aligned} & \sum_{t \in T: t \in T_k \setminus T_l} \sum_{j \in J_t: j \neq k} f_{kj,kl}^t \\ & + \sum_{t \in T: t \notin T_k, t \notin T_l} \sum_{i,j \in J_t: i \neq j} f_{ij,kl}^t \leq (b - |T_l|) x_{kl} \end{aligned} \quad \forall k, l \in J: k \neq l \quad (1.10)$$

$$x_{ij} \in \{0, 1\} \quad \forall i, j \in J_0: i \neq j \quad (1.11)$$

$$\begin{aligned} f_{ij,kl}^t & \geq 0 \quad \forall t \in T, \forall i, j \in J: i \neq j, \\ & \forall k, l \in V_{i,j}^t: k \neq l, \{k, l\} \neq \{i, j\} \end{aligned} \quad (1.12)$$

In the objective function 1.13, they sum all of the tools that a given job in the sequence requires but the immediate previous job does not, and then subtract all of the tools that were kept during 0-blocks in the sequence. That is thus the number of changes performed between jobs, which is added to the number of tools the first job requires.

An important note on the objective function between the three models used in tests for this work: both the SSPMF formulation, which we will see in the next section, and the formulation proposed by this work do not consider in their objective functions the first machine setup as tool switches. This would then always result in a difference of b switches when comparing the models. In order to maintain the same optimal values, we have adapted the CGL objective function so it also does not consider the first machine

setup as counted in the total tool switches. Therefore we used the following objective function for the CGL:

$$\min \sum_{j \in J} |T_j| x_{0j} + \sum_{i,j \in J: i \neq j} \sum_{t \in T_j \cap T_i} (x_{ij} - \sum_{k \in J: k \neq j} \sum_{l \neq i,j} f_{kj,ij}^t) - b \quad (1.13)$$

This change explains some results for which the lower bound obtained via linear relaxation of this problem are negative values, as we are subtracting the capacity from the number of switches.

Constraints 1.3 to 1.5 are classical TSP constraints to control the flow and eliminate sub-tours. In this case, we maintained the classical DFJ sub-tour elimination constraints are the authors use in their paper. In order to be more efficient and avoid having to enumerate every single subset of nodes to create these constraints, we used Gurobipy's callback feature and provided a sub-tour finder function to exploit cutting planes and thus profit the maximum performance of this model.

Constraints 1.6 allow the existence of a 0-block if the path containing it is chosen as the optimal one. Constraints 1.7 to 1.9 guarantee that a tool will only be carried to from one vertex to another if the jobs corresponding to these two vertices are selected to be immediately sequenced in the optimal solution. Constraints 1.10 ensure that the capacity of the machine is always respected considering the tool changes.

1.3 The SSPMF formulation

[9] propose a Multi-Commodity Flow (MCF) based formulation to solve the SSP, which they called SSPMF. The analogy is that the jobs are assigned to nodes in the flow graph, and the tools are the commodities that are transferred through the graph. The SSPMF is based on a graph with $N + 3$ nodes, numbered 0 to $N + 2$, N being the number of jobs of the problem. Node 0 is the origin node, from which all tools begin. Node $N + 1$ is the destination node, to which all tools must flow to, after being used in every tool that requires them. There is also an auxiliar $N + 2$ node that represents a storage location for tools that will still be used in the sequence before flowing to the destination. The set of arcs is composed by four types of arcs:

- Arcs to pass tools for the next job: $(i, i + 1)$, for $i = 0, 1, \dots, N$

1.3. The SSPMF formulation

22

- Arcs to send tools to the storage: $(i, N + 2)$ for $i = 0, 1, \dots, N - 2$;
- Arcs to take tools from the storage: $(N + 2, i)$ for $i = 1, 2, \dots, N - 1$,
- Arcs to send tools to the destination: $(i, N + 1)$ for $i = 1, 2, \dots, N - 1$

They limit the capacity on arc $(i, i + 1)$, for $i = 0, 1, \dots, N - 1$ as C , and the other arcs have unlimited capacity. The only costs for arcs are associated with the arcs that send tools to the storage and arcs that send tools to the destination. Note that sending a tool to the storage means that this tool will sometime be sent to a job node, requiring a tool change in the machine.

The graph for the SSPMF is shown in figure 1.1.

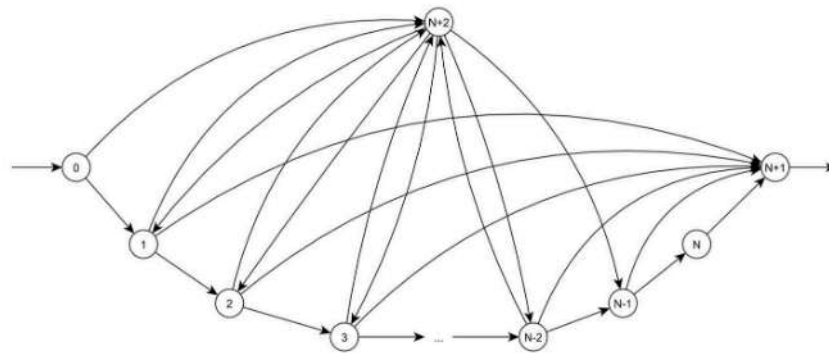


Figure 1.1: Graph from [9]

The SSPMF formulation is defined as follows:

1.3. The SSPMF formulation

23

$$\min \sum_{i=1}^M \sum_{t=1}^{N-1} y_{i(N+1)t} + \sum_{i=1}^M \sum_{t=1}^{N-2} y_{i(N+2)t} \quad (1.14)$$

$$\text{s.t. : } \sum_{k=1}^N x_{ik} = 1, \quad i \in J, \quad (1.15)$$

$$\sum_{i=1}^N x_{ik} = 1, \quad k \in J, \quad (1.16)$$

$$y_{01t} + y_{0(N+2)t} = 1, \quad t \in T, \quad (1.17)$$

$$y_{(i-1)it} + y_{(N+2)it} - y_{i(N+1)t} - y_{i(i+1)t} - y_{i(N+2)t} = 0, \quad t \in T, \quad i = 1, \dots, N-2, \quad (1.18)$$

$$y_{(N-2)(N-1)t} - y_{(N+2)(N-1)t} - y_{(N-1)Nt} - y_{(N-1)(N+1)t} = 0, \quad t \in T, \quad (1.19)$$

$$y_{(N-1)Nt} - y_{N(N+1)t} = 0, \quad t \in T, \quad (1.20)$$

$$\sum_{i=1}^N y_{i(N+1)t} = 1, \quad t \in T, \quad (1.21)$$

$$\sum_{i=0}^{N-2} y_{i(N+2)t} - \sum_{i=1}^{N-1} y_{(N+2)it} = 0, \quad t \in T, \quad (1.22)$$

$$x_{ik} \leq y_{(k-1)kt}, \quad i, k \in J, \quad t \in T_i, \quad (1.23)$$

$$\sum_{t=1}^M y_{(k-1)kt} = b, \quad k \in J, \quad (1.24)$$

$$x_{ik} \in \{0, 1\}, \quad i, k \in J, \quad (1.25)$$

$$y_{i(i+1)t} \in \{0, 1\}, \quad i = 0, \dots, N, \quad t \in T, \quad (1.26)$$

$$y_{i(N+1)t} \in \{0, 1\}, \quad i = 1, \dots, N-1, \quad t \in T, \quad (1.27)$$

$$y_{i(N+2)t} \in \{0, 1\}, \quad i = 0, \dots, N-2, \quad t \in T, \quad (1.28)$$

$$y_{(N+2)it} \in \{0, 1\}, \quad i = 1, \dots, N-1, \quad t \in T. \quad (1.29)$$

The objective function 1.14 minimizes the sum of the number of tools that are sent to the storage (meaning there will be a tool change related to this tool) and the number of tools that are sent to the destination node (which will be all of the tools). Constraints 1.15 and 1.16 ensure that each job will be assigned to exactly one node and each node will have exactly one job assigned to it. Constraints 1.17 to 1.22 are the flow conservation constraints that ensure that all tools start from origin node, end at destination node, and that any tool that enters a job or storage node will flow out of it. Constraints 1.23 ensures that all of the tools required by a given job will flow into the node for which the job was assigned. Finally, constraints 1.24 ensures that the number of tools flowing between two consecutive nodes is exactly the capacity of the machine.

1.4 Heuristics and Metaheuristics

Even the best exact formulations developed so far are evaluated using their linear relaxation bounds, as larger instances of the SSP are very hard to treat. Due to its difficult nature to be solved optimally for these larger instances, the SSP was tackled by many authors using heuristic methods and metaheuristics, aiming good solution generation in feasible computer time. The use of heuristic methods started early with [19], when they proposed a Greedy Perturbation algorithm in which a job processing sequence was created, then evaluated using the KTNS algorithm and finally perturbed to generate neighbor solutions that improved the objective function. Although inefficient for larger instances, this heuristic marked one the first attempts to find intelligent ways to obtain upper bounds for the SSP.

As this work will partly focus on heuristic methods, it is important to state some important heuristic methods for the SSP. Many heuristic methods were develop modeling the sequencing part of the SSP as the TSP. One of the most relevant was created by [11]: the GENIUS method, composed of a GENI constructive heuristic and an US improvement heuristic. The sequencing part of the problem was based on the TSP, and the GENI heuristic creates the TSP tour iteratively, rearranging parts of the sequence on-the-go. US tries to improve an already complete route by removing a vertice from the sequence and reinserting it elsewhere. Many other authors used the TSP-based model to create their heuristics, as compared by [17].

Some authors created interesting graph-based heuristics where the nodes are tools instead of usual job nodes. This graph is known as either SSP graph or MOSP graph [9]. In [6] constructive heuristic, two tool nodes are linked to each other if one specific job requires both tools. Those two tool nodes may have more than one link between them if more than one job requires both tools. The constructive heuristic selects the node with the lowest degree and inserts in the sequence all the jobs that require the tool from that node. This process is repeated until all jobs have been sequenced. They then used an Iterated Local Search (ILS) procedure to further improve the current solution, obtaining good results.

Many authors proposed metaheuristic methods to solve the SSP, aiming for more generalist approaches and benefiting from the large literature regarding those methods. [1] proposed a Tabu Search heuristic, for which the neighbor generating scheme was based on either random swapping or random block insertion, obtaining promising results.

[2] compared multiple variations of memetic algorithms, simulated annealing algorithms, hill climbing algorithms, tabu searches, beam searches and a genetic algorithm. In the computational tests, the memetic algorithm embedded with the hill climbing search presented the best results.

[5] proposed a hybrid heuristic based on the Biased Random Key Genetic Algorithm (BRKGA) and the Clustering Search (CS). The CS procedure intensifies the search for good solutions in regions in the search space that seem promising. The BRKGA is a well-known genetic algorithm adaptation that leverages the feature of random keys of always generating feasible solutions from offspring formed by crossover. They used a Variable Neighborhood Descent (VND) as the local search heuristic.

[15] also used a tool-based graph, but they assign weights to the edges instead of parallel edges. They start by creating a tool sequence based on this graph, then they use a greedy procedure to derive a job sequence from the tool sequence and thus obtain an initial solution. They also proposed an ILS method to improve the solutions.

Finally, one of the last big contributions in heuristic and metaheuristic methods to solve the SSP was introduced by [14]. They have proposed a hybrid genetic search. They start with a population of solutions that were randomly created and then improved by a local search in each one. They then proceed to iteratively select pairs of parents in order to generate an offspring solution and further improve it by the local search procedure. A survivor phase is applied and the procedure is repeated until a local maxima is achieved. They reported the best heuristic results so far in the field and will be used as benchmark for this work.

Chapter 2

Proposed model and Preliminary Results

As explained in the introduction, exact mathematical models for the SSP have been evaluated regarding its lower bounds provided by linear relaxation in larger instances. In this section, we will present the new proposed formulation due to [7], compare its results to other formulations regarding the lower bounds, and explore a feature of the new formulation: the use of heuristics to generate tool repository configurations that are likely to be in an optimal solution.

Let $\mathcal{C} = \{C_1, C_2, \dots, C_p\}$ be the collection of all possible tool repository configurations. Let us consider the complete graph K_p , where each vertex corresponds to a configuration. Let $d : E(K_p) \rightarrow \mathbb{R}$ be a distance function between two given configurations. In this sense, $d(i, j)$ denotes the number of tool switches required to obtain configuration C_j from configuration C_i , that is,

$$d(i, j) = b - |C_i \cap C_j|.$$

Let $H_j \subseteq V(K_p)$ denote the subset (cluster) of configurations that are capable of processing job $j \in J$. That is,

$$H_j = \{C_i \in \mathcal{C} \mid T_j \subseteq C_i\}.$$

Solving the Tool Switching Problem can now be seen as the problem of finding the shortest Hamiltonian cycle in K_p visiting a collection of vertices $X \subseteq V(K_p)$ with the property that at least one vertex from each cluster H_j , for $j \in J$, is visited (i.e., is contained in X).

We can thus identify two NP-hard problems within the Tool Switching Problem:

- **Covering Problem:** The clusters H_j must be covered by X .
- **Traveling Salesman Problem:** Once the covering is defined, a TSP must be solved to determine the best ordering of jobs.

The Tool Switching Problem can thus be interpreted as a combination of the Covering and TSP problems.

The Generalized Traveling Salesman Problem (GTSP) is a well-known combinatorial problem [16]. It can be defined as follows. Given an undirected graph G whose vertices are divided into a given number of clusters, denoted by $V_1, \dots, V_n$, the GTSP searches for the shortest Hamiltonian cycle visiting a collection of vertices with the property that exactly one vertex from each cluster is visited. In this sense, GTSP can be seen as a particular case of Tool Switching Problem where the clusters H_j are all disjoint.

2.1 A non-compact formulation for the Tool Switching Problem

Let $y(v)$ denote a binary variable equal to 1 if configuration $v \in V(K_p)$ is used to process some job, and equal to 0 otherwise. Let $x(u, v)$ denote a binary variable equal to 1 if configuration $v \in V(K_p)$ is selected immediately after configuration $u \in V(K_p)$ in the optimal sequence of configurations and equal to 0 otherwise. The Tool Switching Problem can thus be modeled as follows:

$$\min \sum_{u \in V} \sum_{v \in V} d(u, v) x(u, v) \quad (2.1)$$

$$\text{s.t. :} \quad (2.2)$$

$$\sum_{v \in H_j} y(v) \geq 1 \quad \forall j \in J, \quad (2.3)$$

$$\sum_{u \in V} x(u, v) = y(v) \quad \forall v \in V, \quad (2.4)$$

$$\sum_{v \in V} x(v, u) = y(u) \quad \forall u \in V, \quad (2.5)$$

$$\sum_{u \in S} \sum_{v \in S} x(u, v) \leq \sum_{v \in S} y(v) - 1, \quad \forall S \subseteq V, 2 \leq |S| \leq |V| - 1, \quad (2.6)$$

$$x(u, v) \in \{0, 1\}, \quad \forall u \in V, v \in V, \quad (2.7)$$

$$y(v) \in \{0, 1\} \quad \forall v \in V. \quad (2.8)$$

The formulation described above is clearly non-compact as there are exponentially many possible configurations. Indeed, the number of possible configurations is given by

$$p = \binom{m}{b} = \frac{m!}{b!(m-b)!}. \quad (2.9)$$

Therefore the size of V is exponential in the size of T , which renders this formulation untractable even for medium-sized instances.

2.2 Proposed work

The instances used for our preliminary comparative tests were the datA group of instances, a group of small instances for which we were able to run the complete proposed model with all of its configurations. This group of instances is known in the literature as been proposed by [4], and we will approach the other groups in the results section of this work.

In particular, this preliminary tests used the instances of the datasets datA1, datA2, datA3 and datA4, as all 40 instances of these datasets have 10 jobs and 10 tools each. The magazine capacity for the four groups are 4, 5, 6 and 7, respectively, which gives us a maximum of 252 nodes in the case of capacity = 5. As this is a relatively low amount of nodes for nowadays commercial solvers and RAM usage, it was possible to solve even the new proposed model with all of its nodes created, which guarantees optimality. Thus we decided to use them to solve all models optimally and linearly relaxed for those 40 instances.

The results were promising in regards to the LP values and thus motivated us to keep exploring this formulation and try to tackle its difficulties for larger instances with a heuristic method.

For this preliminary work, we compared the Formulation 5 from [4] (the one they report had the best results), hereinafter named CGL formulation, the SSPMF from [9] and the new formulation proposed by [7].

For all of the results of this section, the new formulation results were obtained by enumerating every possible magazine configuration for each instance. This was only possible due to the small sets of tools and low value of magazine capacity of those instances.

Linear relaxation values are widely used as a good metric of how tight the formulation is and thus an indicator of expected good performance for larger instances. Table 1 shows the linear relaxation (LP) values obtained for all three models, the runtime in seconds for each of the LP runs and also the optimal value (column Opt) for comparison, for each of the 40 instances of small instances group dataA.

Note that the SSPMF always provides $M - b$ as the Linear Relaxation optimal value. As [9] explains in their paper, $M - b$ is a trivial lower bound for SSP: assuming that b tools begin in the magazine and that all M tools will at some point be required to process at least one job, we can affirm that all the tools that are left out in the beginning must enter the machine at some moment later, which leads to at least $M - b$ tool switches.

All three models were solved in less than 1 second of run time for the linear relaxation for every instance in dataA, thus we did not include the run time for this table. In regards to the obtained lower bounds, the trivial bound $M - b$ obtained by SSPMF was the best bound for all 30 instances from subgroups dataA2, dataA3 and dataA4. For tighter instances from dataA1, however, it was able to obtain the best bound for only 6 instances (one tie with the new formulation). CGL model found the best bound in 1 instance and the new formulation in 4 instances.

Here we state again why we have some negative lower bounds for the CGL model: the original model from [4] did consider that the magazine setup at the start of the sequence counts as tool switches. Thus all of the obtained values with it will be b switches above what it would be when translated to the other models (New and SSPMF do not consider the setup as tool switches). Hence we opted for subtracting from the objective function the value of the magazine capacity for each instance. This effect is also explained in [9], although they decided to sum the capacity in their model for the results.

We also have in table 1 the runtime for each model for the complete MILP formulations in order to find the optimal solution for the same instances. All times are in seconds and all of the models found the optimal solution for every instance of this table.

We can observe clearly that SSPMF model was the fastest for almost all of the 40 small instances, specially in subgroups dataA2, dataA3 and dataA4, for which the magazine capacity is increased. The lower bounds obtained by both CGL and the new proposed model is worse when the capacity increases, indicating that they perform better in tighter instances. The new proposed formulation performed well for most instances, only once exceeding one minute of run time. The CGL model was the model that took the most time to solve by a large margin for every instance of the three cited subgroups.

2.2. Proposed work

30

Instance	Opt	SSPMF		CGL		New Model	
		MIP time	LP	MIP time	LP	MIP time	LP
A1-1	10	15	6	0	8	1	7
A1-10	8	5	6	1	6	1	6
A1-2	7	1	6	5	3	2	5
A1-3	8	3	6	5	4	4	5
A1-4	9	8	6	1	6	3	6
A1-5	9	9	6	0	6	1	7
A1-6	10	14	6	1	7	1	7
A1-7	8	3	6	4	4	2	5
A1-8	7	0	6	3	3	2	5
A1-9	9	5	6	0	7	0	7
A2-1	6	1	5	23	1	11	4
A2-10	6	2	5	45	0	22	3
A2-2	5	0	5	85	-1	7	3
A2-3	5	0	5	54	-1	39	3
A2-4	6	0	5	59	-1	26	3
A2-5	6	1	5	25	1	6	4
A2-6	7	17	5	68	0	31	4
A2-7	6	1	5	94	-1	127	3
A2-8	5	0	5	98	-1	8	3
A2-9	6	1	5	12	1	5	4
A3-1	4	0	4	115	-3	4	2
A3-10	4	0	4	250	-3	39	3
A3-2	4	0	4	417	-3	28	2
A3-3	4	0	4	369	-3	24	2
A3-4	4	0	4	231	-3	8	2
A3-5	4	0	4	142	-3	4	2
A3-6	5	3	4	408	-3	21	2
A3-7	4	0	4	313	-4	29	2
A3-8	4	0	4	435	-3	28	2
A3-9	4	0	4	112	-2	8	2
A4-1	3	0	3	523	-4	1	2
A4-10	3	0	3	715	-4	6	2
A4-2	3	0	3	738	-4	4	1
A4-3	3	0	3	628	-4	7	1
A4-4	3	0	3	592	-4	3	2
A4-5	3	0	3	642	-5	1	2
A4-6	3	0	3	666	-4	2	2
A4-7	3	0	3	888	-5	3	2
A4-8	3	0	3	811	-4	2	1
A4-9	3	0	3	401	-4	1	2

Table 1: Optimal value, complete MIP models runtime (s) and linear relaxation values for the three formulations for small instances (datA group).

Chapter 3

Heuristics based on the proposed model for larger instances

The promising results for the new formulation indicate that it may be tighter than the best models known to us, and thus deserves a deeper work aiming at smart and efficient ways to create good tool configurations to make it possible to run the model. Hence in this work we have developed a specific heuristic method that leverages some problem features in order to run the new formulation with fewer but useful tool configurations sets, obtaining upper bounds to the SSP.

In this section, we will explore the motivations that validated the possible positive impact of this heuristic method, some basic concepts that were used in the construction of the heuristic method and finally we will explore every step of the heuristic in order to explain how it works and what it aims to tackle.

3.1 Motivation for configuration generation heuristic methods

The new formulation naturally tends to perform better when fewer configurations are provided. To understand the impact of the amount of configurations provided for the formulation, we conducted the following experiment: first, the formulation is solved optimally for a small instance, by generating $\mathcal{C}$, in other words, all possible magazine configurations for the instance. We then obtain Q , the set of configurations that were used in the optimal solution. Following that, we start a loop that solves the formulation k times, and at each iteration i provides to the formulation a configuration set that contains all of the configurations in Q and also $|\mathcal{C}| \cdot \frac{i}{k}$ randomly selected configurations from $\mathcal{C} \setminus Q$.

Figure 3.1 shows the time in seconds used to optimally solve each instance by the number of extra configurations $|\mathcal{C}| \cdot \frac{i}{k}$ provided for it. These are the 10 original small instances for which $N = 10$, $M = 10$ and $b = 4$. The results indicate that the formulation works extremely well when given a smaller set of configurations. Each instance took less than one second to solve when only Q was provided to the formulation, and it kept good performance with a few extra configurations, enhancing the motivation to create good heuristic methods for this configuration generation.

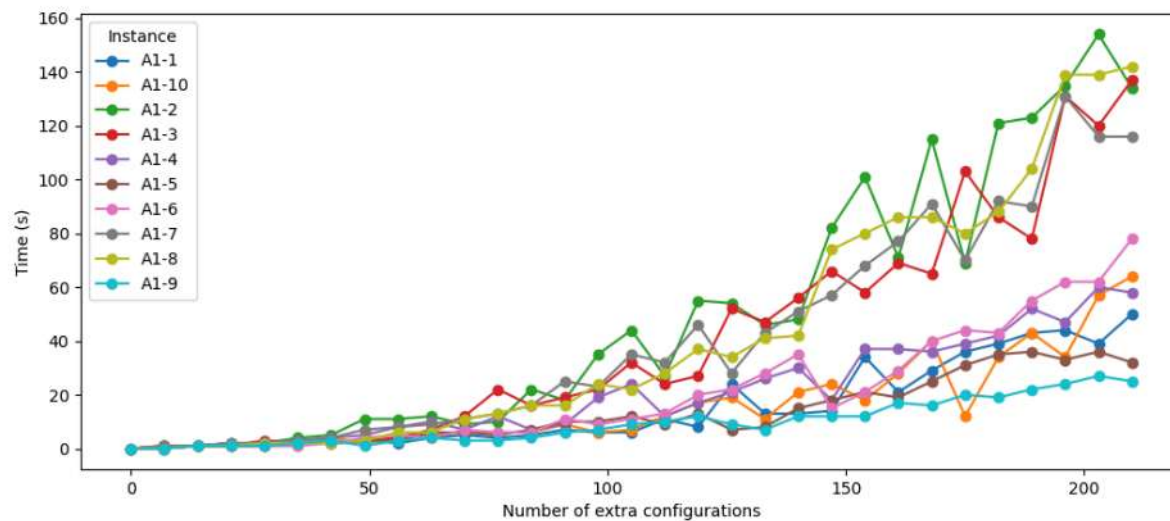


Figure 3.1: Computational time (s) to solve new formulation for 10 small instances by number of extra configurations provided.

3.2 General guidelines to find good configuration sets

The goal of this section is to explore some ideas of configuration generation heuristics that were used in this work. It is then important to state the goal of these heuristic methods.

Consider the problem described in the Introduction, the formulation proposed by [7] and the set of all possible configurations described in section 2. The central idea of this heuristic method is to generate the best possible subset of magazine configurations such that:

- The optimal solution for the partial problem (using this set of configurations created by the heuristic) is as close as possible to the optimal solution of the complete

problem, considering all combinations of tools for the magazine capacity, meaning it contains configurations that are used to process many jobs and thus reduces tool switching needs;

- It is still small enough so that solving the new formulation optimally for this subset is computationally cheap (if it is so cheap that we are able to do it multiple times with multiple starts, even better).

We must then clarify some basic requirements in order to generate any given magazine configuration.

A magazine configuration is defined as a set of b tools. Naturally, it makes no sense to have a configuration with size less than the magazine capacity, as any tool that can be kept might be useful later in the sequence and thus reduce tool switches. In other words, all magazine configurations must have exactly b tools.

For some other formulations, this feature of magazine configurations might be "hidden". Take for instance the CGL formulation: if the first job j in the optimal sequence has a toolset of size $|T_j| < b$, then you will not know which are the $b - |T_j|$ tools that complement the first magazine configuration until you check the differences from the first job to the second. Nevertheless, it is always possible to derive which were the magazine configurations (or at least an optimal sequence of magazine configurations) for the other formulations as well.

It is also pointless to create a configuration that does not contain at least one complete tool set T_j for one job $j \in J$, as it would be necessary to switch another tool to process the following job and thus it would be the same as jumping straight to the next configuration. This might not be relevant in terms of practical applications in some cases, but it is very important for the proposed formulation as those unused configurations add nodes to the graph and could be replaced by actual useful nodes.

Another useful concept in this regard is the distribution of the number of fully satisfied jobs by each configuration (excluding those considered unused) for each instance, which gives us an understanding of how many configurations may be really important to create when aiming to gather multiple jobs in one single configuration.

The configuration generation for the hereby described heuristics have two main objectives:

- Maximize the amount of configurations that satisfy the tool requirements for two or more jobs at the same time. In fact, the more jobs one single configuration may

satisfy, the better. It is not guarantee of optimality but there is a strong rational to it: the very thing that reduces the amount of tool switching is the existence of configurations that satisfy many jobs at once;

- Smooth the transition between very different tool sets required by different jobs. Here, the idea is that we should not solely focus on exploring configurations inside clusters of similar jobs, but also try to find ways to connect the clusters in a way that fewer tool switches are necessary.

When thinking of a constructive heuristic, two main ideas appear naturally: start building a configuration from one specific tool or start building a configuration from one specific job, or rather the required tools for one specific job. When creating the heuristics for this work, we decided to focus on the latter, because any heuristic that starts from one specific tool would require constant checking to make sure any created configuration holds the assumption that it contains a complete tool set required by at least one job. Starting from a specific job, on the other hand, guarantees that at least one job is satisfied by any created configuration and thus saves computational effort.

There is also one specific case worth mentioning: any job j for which $|T_j| = b$ provides an already complete magazine configuration. The heuristic proposed for this work will already treat this case right away by creating the configuration that is equal to T_j . However, this job will not be removed from the rest of the logic, as this configuration might impact others depending on the defined logic. An interesting point about this kind of job is that the number of jobs for which $|T_j| = b$ is a lower bound to the SSP, as anytime one of them comes up at least one tool switch is certainly mandatory.

3.3 Summary of the heuristic method

This section describes the heuristic method created to tackle the SSP by using the new proposed formulation. In this section, we will first explain in a short step list the main idea of the algorithm and we will then provide further detail of every step in the following sections.

The heuristic method is composed by a multi-start phase where the new formulation is solved many times. The only difference between these runs is the subset of magazine configurations provided for the model. For medium and large instances, as explained in

previous sections, it is impossible to enumerate all magazine configurations and provide it to the model, as its number grows exponentially. The idea is then to try to enumerate only the configurations that would be used by the optimal solution to the complete problem.

To find those configurations, however, many different - and sometimes opposite - strategies might be employed. That is why we propose for this heuristic a multi-start, trying to leverage the multiple factor that may generate different sets of configurations.

The results of each iteration for the multi-start also provide some advanced knowledge of which are actually good and useful configurations for each instance. For this reason, at the end of the multi-start phase, we also add an improvement phase that uses previously created configurations as an already filtered pool and solves the formulation one last time.

The heuristic method proposed by this work comprises the following steps that are completed sequentially:

1. **Definition of Multi-Start iterations:** in this first step, we define the set of parameters and strategies that will be used in the multiple starts of the Phase 1 of the heuristic.
2. **Multi-Start configuration generation phase:** This step is repeated several times, and each iteration is completely separated from the others.
 - a) **Generation of base configurations:** We begin by creating exactly one configuration that satisfies the tool requirements for each job of the instance, hence we guarantee feasibility. This configuration is created using the Configuration Creator 1, an algorithm that will be explained in the next sections.
 - b) **Generation of extra configurations:** Having a maximum amount of configurations that we can create in order to keep the iterations fast, we then proceed to create extra configurations until the maximum amount is reached. These configurations are created using the Configuration Creator 2, another algorithm that will be explained in the next sections.
 - c) **Optimization:** Using the magazine configurations subset created by the previous procedures, we run the optimization of the new formulation using this subset as the nodes in V . The optimal solution returned by this step is of course an upper bound to the complete problem, as only a subset of the configurations was used.
 - d) **KTNS improvement:** We then try to improve the upper bound obtained in the previous procedure by taking the job sequence of the obtained solution and applying the KTNS policy to it. If it improves the current number of

tool switches, we update the current best upper bound. We also keep the configurations used in the optimal solution for each iteration.

3. **Mixing and improving configurations phase:** This step is performed only once, at the end of all of the previous steps.

- a) **Selection of base configurations:** Having the pool of configurations used in the best upper bound solutions for every iteration of Phase 1, we first ensure every job will be satisfied by at least one configuration by randomly selecting a configuration in the pool of configurations that satisfy every job.
- b) **Selection of extra configurations:** We then assign weights to remaining configurations based on how many jobs they satisfy and choose configurations weighted-randomly.
- c) **Optimization:** Using the magazine configurations subset selected by the previous procedures, we run the optimization of the new formulation using this subset as the nodes in V . The optimal solution returned by this step is of course an upper bound to the complete problem, as only a subset of the configurations was used.
- d) **KTNS improvement:** We then try to improve the upper bound obtained in the previous procedure by taking the job sequence from the solution and applying the KTNS policy to it. If it improves the current number of tool switches, we update the obtained upper bound.

The implementation of this heuristic method was fully done using Python 3.12. We opted for a class structure that could capture the needs of the problem specifics. We created initially three main classes for the mathematical objects:

- A class Job to store data about each job:
 - A set of required Tools;
 - A set of satisfying Configurations;
 - The maximum number of satisfying configurations: $\binom{M-|T_j|}{b-|T_j|}$;
 - A counter for how many satisfying configurations are left uncreated, equal to the maximum number of satisfying configurations minus the size of the set of satisfying Configurations.
- A class Tool to store data about each tool:
 - A set of Jobs that require the Tool;
 - A set of Configurations that include the Tool;

3.4. Step 1: Defining parameters and strategies for the multi-start iterations 37

- The Frequency metric for this tool, i.e. the size of the set of Jobs that require the Tool.
- A class Configuration to store data about each created configuration:
 - The set of b Tools that defines the Configuration;
 - The set Jobs for which their tool requirements is satisfied by the Configuration.

3.4 Step 1: Defining parameters and strategies for the multi-start iterations

Defining the parameters and strategies of each iteration of the multi-start is a key step, as we aim to find a different set of magazine configurations each time, allowing us to explore different parts of the complete pool.

Between iterations of the multi-start, there are three features that have to be defined and that will highly impact how the heuristic will create the configurations for each of the iterations. Two of them are strategies: Next Job Strategies and Next Tool Strategies, methods that define how we will prioritize jobs or tools in the configuration creation. The third is a parameter of randomness, the Random Factor, to ensure we explore different configurations.

Each iteration of the multi-start will have exactly one value of Random Factor, one Next Job Strategy and one Next Tool Strategy. For the experiments of this work, we have used all of the combinatorial possibilities of two values of Random Factor (10% and 30%), three Next Job Strategies and two Next Tool Strategies. Table 2 shows all of the 12 combinations generated for the 12 runs of the multi-start.

Take for instance the multi-start run number 1. For every one of the configurations that will be created for this iteration of the multi-start, we will define the next job to be used as base by using the Least Covered next job strategy about 90% of the time and a random selector about 10% of the time. When a configuration is still incomplete and we need to determine which will be the next tool to enter the configuration, we will use the Most Frequent next tool strategy about 90% of the time and a random selector about 10% of the time.

Multi-Start Run	Random Factor	Next Job Strategy	Next Tool Strategy
1	0.1	Least Covered	Most Frequent
2	0.1	Least Covered	Most Joint-Frequent
3	0.1	Most Available Configs	Most Frequent
4	0.1	Most Available Configs	Most Joint-Frequent
5	0.1	Lowest Config Share	Most Frequent
6	0.1	Lowest Config Share	Most Joint-Frequent
7	0.3	Least Covered	Most Frequent
8	0.3	Least Covered	Most Joint-Frequent
9	0.3	Most Available Configs	Most Frequent
10	0.3	Most Available Configs	Most Joint-Frequent
11	0.3	Lowest Config Share	Most Frequent
12	0.3	Lowest Config Share	Most Joint-Frequent

Table 2: Value of Random Factor and Next Job and Next Tool strategies elected for each of the 12 multi-start runs that will be performed for each instance.

3.4.1 Next Job Strategies

As previously explored, we decided to create every configuration by starting with one specific job of the instance. We begin of course by creating for each job one configuration that satisfies its tool requirements. After that, we face the challenge of defining which job should be the base for the next configuration. As this is a crucial decision that could lead to very different results, three Next Job Strategies were developed and implemented, and each iteration of the multi-start uses exactly one as its strategy.

Note that we will mostly try to create configurations capable of satisfying many jobs’ tool requirement sets. This means that one configuration created using job i as base might also satisfy other jobs, say j and k . In this case, all three jobs i, j, k will have this configuration in their satisfying configurations set and the counter of remaining uncreated satisfying configurations of all three jobs will be deducted in 1.

3.4.1.1 Next Job Strategy: Random

The simplest idea of all is of course just selecting any random job for which there is still at least one remaining uncreated satisfying configuration and its tool requirement set size is less than the magazine capacity, so that there is a chance we actually created a new configuration and not a copy of one that already exists. This logic is shown in algorithm

3.4. Step 1: Defining parameters and strategies for the multi-start iterations

39

1.

Algorithm 1 Get Random Job

```

function GETRANDOMJOB
   $\mathcal{A} \leftarrow \{j \in \text{jobs} \mid j.\text{remConfigs} > 0 \text{ and } |j.\text{reqTools}| < \text{capacity}\}$ 
  if  $\mathcal{A} \neq \emptyset$  then
    return random element from  $\mathcal{A}$ 
  else
    return random element from jobs
  end if
end function

```

3.4.1.2 Next Job Strategy: Least Covered Job

For this strategy, we find the job for which there is the fewest number of created configurations that satisfy its tool requirements, by accessing its set of satisfying configurations as shown in algorithm 2.

Algorithm 2 Find Least Covered Job

```

1: function GETLEASTCOVEREDJOB
2:    $\mathcal{A} \leftarrow \emptyset$  ▷ Set of available jobs
3:   for all  $job \in \text{jobs}$  do
4:     if  $job.\text{remConfigs} > 0$  and  $|job.\text{reqTools}| < \text{capacity}$  then
5:        $\mathcal{A} \leftarrow \mathcal{A} \cup \{job\}$ 
6:     end if
7:   end for
8:   if  $|\mathcal{A}| = 0$  then
9:     return GETRANDOMJOB
10:  else
11:    return  $\arg \min_{j \in \mathcal{A}} (|j.\text{satisfyingConfigs}|)$ 
12:  end if
13: end function

```

This strategy tries to balance the number of configurations created using each job as a base equally between all jobs. Naturally, for most cases the jobs will have different tool requirement set sizes, and thus this balance will not be perfect if enough iterations are performed. However, this strategy ensures that if a job i has more than 2 configurations more as base than any job j , this means all of the possible satisfying configurations for job j are already created.

3.4. Step 1: Defining parameters and strategies for the multi-start iterations

40

For example, if we have $b = 4$, $|M| = 10$ and two jobs, job i for which $|T_i| = 3$ (7 satisfying configurations) and job j for which $|T_j| = 2$ (56 satisfying configurations), we would alternate between both jobs until all 7 configurations using job i as base were created, and only then proceed to use j as base until the maximum number of configurations is reached.

3.4.1.3 Next Job Strategy: Most Available Configurations

For this strategy, we find the job for which there is the largest number of uncreated satisfying configurations, by accessing its internal counter as shown in algorithm 3.

Algorithm 3 Find Job with Most Available Configurations

```

1: function GETMOSTAVAILABLECONFIGSJOB
2:    $bestJob \leftarrow \text{null}$ 
3:    $maxConfigs \leftarrow -1$ 
4:   for all  $job \in \text{jobs}$  do
5:     if  $job.remainingConfigs > maxConfigs$  then
6:        $maxConfigs \leftarrow job.remainingConfigs$ 
7:        $bestJob \leftarrow job$ 
8:     end if
9:   end for
10:  return  $bestJob$ 
11: end function

```

This strategy tries to tackle instances for which job have very different sizes of tool requirement sets. Jobs with smaller tool requirement sets will be selected more often, as they occupy a smaller portion of the magazine capacity and thus might provide a more variable set of magazine configurations when selected to be the base.

For example, if we have $b = 4$, $|M| = 10$ and two jobs, job i for which $|T_i| = 3$ (7 satisfying configurations) and job j for which $|T_j| = 2$ (56 satisfying configurations), we would try to create 49 configurations using j as base to only then start creating configurations using i as base.

3.4. Step 1: Defining parameters and strategies for the multi-start iterations 41

3.4.1.4 Next Job Strategy: Lowest Share of Satisfying Configurations

For this strategy, we calculate for each job the share of satisfying configurations already created for the job, by performing the division of the size of created satisfying configurations for the job by the counter of remaining uncreated configurations as shown in algorithm 4.

Algorithm 4 Find Job with Most Available Configurations

```

1: function GETLOWESTSHAREJOB
2:    $bestJob \leftarrow \text{null}$ 
3:    $lowestShare \leftarrow 1$ 
4:   for all  $job \in \text{jobs}$  do
5:      $share \leftarrow |job.satisfyingConfigs| / job.maxConfigs$ 
6:     if  $share < lowestShare$  then
7:        $lowestShare \leftarrow share$ 
8:        $bestJob \leftarrow job$ 
9:     end if
10:  end for
11:  return  $bestJob$ 
12: end function

```

This strategy tries to balance both the strengths of Least Covered Job and Most Available Configurations. It ensures that the jobs with more available configurations will be prioritized

For example, if we have $b = 4$, $|M| = 10$ and two jobs, job i for which $|T_i| = 3$ (7 satisfying configurations) and job j for which $|T_j| = 2$ (56 satisfying configurations), every 8 new configurations created using j as base would then give space for 1 new configuration created using i as base.

3.4.2 Next Tool Strategies

For some parts of the algorithm, we will try to find configurations that satisfy more than one job and hence better profit the magazine capacity by adding more and more jobs to it. However, many times we will not be able to find full magazine configurations (with size exactly b) by just putting jobs together. Sometimes, we will reach a point

where the configuration we are building still has vacant slots in it but no other job's tool requirement set could be added without violating the magazine capacity.

For this scenario, we must also have strategies to select the next tool to enter the configuration up until we reach a configuration of size b . We then created two Next Tool Strategies that are based in the transition smoothing rational described previously. At this point, as we have already reached the most jobs we can satisfy by the configuration, we turn our attention to other jobs that might arise in the optimal sequence and thus keeping useful tools in this configuration might pay off later in the optimization.

Hence the main idea for the next tool selection is based on trying to find tools that have some level with synergy with one another. We then use two similarity metrics in order to find i) tools that are used by a large number of jobs and thus improve our chances of reusing this tool in jobs that will come next in the sequence and ii) tools that are used together with the tools that are already in a configuration being created and thus improve our chances of having less switches to next configurations in the route. These concepts are respectively the Frequency of a tool and the Joint Frequency between a tool and a group of tools.

- **Frequency:** for a tool t , it is the number of jobs that require t to be processed.
- **Joint frequency:** for this pair of tools (t, s) , it is the number of jobs that require both tools to be processed. We can also calculate the joint frequency of a tool t in relation to a group of tools F as the sum of the joint frequency of every pair formed by t and each tool in F separately.

3.4.2.1 Next Tool Strategy: Random

In the same manner as for the Next Job Strategies, the simplest idea is to just select randomly any tool of the instance. This is even simpler for tool selecting as we do not even have to check if it is saturated. The idea is shown in algorithm 5.

Algorithm 5 Get Random Tool

```

function GETRANDOMTOOL( $\mathcal{T}_{available}$ )
    return random element from  $\mathcal{T}_{available}$            ▷ Selected uniformly at random
end function

```

3.4.2.2 Next Tool Strategy: Most Frequent Tool

For this strategy, we select from the available tools (which are the tools not yet in the configuration being created) exactly one tool randomly using weights. The random factor here is important so we can create a wider number of different configurations, whilst maintaining some rational to the decision. The weights are simply for each tool its Frequency metric, i.e. the number of jobs that required that tool in order to be processed. This logic is shown in algorithm 6.

Algorithm 6 Get Most Frequent Tool

```

function GETMOSTFREQUENTTOOL( $\mathcal{T}_{available}$ )
     $\triangleright$  Select a tool based on its frequency weight
    Choose  $t \in \mathcal{T}_{available}$  with probability:
        
$$P(t) = \frac{\text{freq}(t)}{\sum_{t' \in \mathcal{T}_{available}} \text{freq}(t')}$$

    return  $t$ 
end function
  
```

This strategy favors the tools that are most likely to be used if we placed any random job right next to this configuration in the sequence, thus aiming to reduce tool switches in the most general case.

3.4.2.3 Next Tool Strategy: Most Joint-Frequent Tool

For this strategy, we calculate for each available tool (which are the tools not yet in the configuration being created) its Joint-Frequency metric, i.e. the number of jobs that use both the candidate tool and any of the tools that are already in the configuration. This logic is shown in algorithm 7.

Algorithm 7 Get Most Joint Frequent Tool

```

function GETMOSTJOINTFREQUENTTOOL( $\mathcal{T}_{available}, \mathcal{T}_{current}$ )
     $\triangleright$  Identify the tool that maximizes joint coverage with the current set
    return  $\arg \max_{t \in \mathcal{T}_{available}} \left| \{j \in \text{jobs} \mid (\mathcal{T}_{current} \cup \{t\}) \subseteq j.\text{reqTools}\} \right|$ 
end function
  
```

3.4. Step 1: Defining parameters and strategies for the multi-start iterations 44

This strategy tries to anticipate other jobs that might come next in the sequence and that have similar tool requirement sets are those that are satisfied by the tools that are in the configuration.

3.4.3 Maximum Amount of Configurations

When creating different configurations for the optimization with the new formulation, one must decide when it is enough configurations. Adding one extra configuration is of course a tradeoff between gaining one more node in the graph and thus have a chance of improving the obtained upper bound by reducing a tool switch versus adding to the combinatorial number of variables and constraints of the problem and thus spending more computationally.

For this work, we defined a hard value, 100 configurations, as the maximum amount of configurations provided to the model in each iteration. This decision was base purely on the fact that the heuristic must run the model multiple times for the multi-start and also it must keep a low computational cost compared to the exact formulations benchmark.

3.4.4 Maximum Amount of Multi-Start Iterations

Every multi-start iteration of the heuristic will cost some time, and as the instances grow, even with relatively few nodes on the graph, the formulation takes some time to be solved. This means of course that we have a limited amount of multi-start iterations and thus this is also an input parameter for the heuristic. For this work, we have defined 5200 seconds as the upper limit, and this time is split between all multi-start and improvement phases.

3.5 Step 2: Multi-Start Configuration Generation Phase

The previous step will determine all of the different parameters and strategies that we will use to run the multi-start iterations. In this section, we will explore specifically how one multi-start iteration is performed in the heuristic.

Having defined all of the essential variable parameters and strategies, each iteration will receive a set of parameters and strategies containing exactly one option for: Next Job Strategy, Next Tool Strategy, Random Factor, Maximum Amount of Configurations and a time limit for the optimization phase.

We then take those parameters and strategies for the current iteration and proceed to the configuration generation. This configuration generation starts by creating at least one configuration that satisfies the tool requirements for every job in the instance, hereby named base configurations, and proceeds to fill the remaining configuration spots with configurations created starting from one specific job, selected using the Next Job Strategy of the iteration. Those configurations created in the second part are hereby named extra configurations.

3.5.1 Generation of base configurations

In this step of the multi-start, we ensure the iteration will surely provide a feasible solution to the problem. As we provide the model only a subset of all possible magazine configurations, if we skip this phase, there is the risk of creating a subset of magazine configurations for which no configuration satisfied one given job.

To create new configurations, we have initially develop the following method:

3.5.1.1 Multi-job Configuration Creator

Our first and main idea is to find groups of jobs for which the union of the tool requirement sets of those jobs has a smaller size than b . In other words, groups of jobs that can be processed all at once, with only one magazine configuration.

As explored previously, having more than one job being satisfied by the same configuration points to the direction of reducing tool switches. That is why this Configuration Creator will be prioritized in the heuristic.

The Multi-job Configuration Creator works as follows. One specific job is received and used as base, i.e. its required tools are fixed in the configuration to be created. While there are vacant slots, i.e. the size of the configuration is less than b tools, we randomly select another job and check whether we can add all of the tools this new job requires without violating the magazine capacity. If yes, we add the tools. If no, we simply jump to the next random job. We repeat this procedure until we reach exactly b tools in the configuration or if no more jobs are available.

If we ended the procedure described above with less than b tools in the configuration, we proceed to select and add one tool at a time using the iteration's Next Tool Strategy, until a magazine configuration of size b is reached.

The logic of the Multi-job Configuration Creator is also shown in algorithm 8.

By using the Multi-job Configuration Creator multiple times, we might create many configurations that satisfy the same group of jobs, while having some level of variation in regards to which groups are created, depending on the base job and the random selection of the other jobs.

3.5.2 Generation of extra configurations

After making sure every job will be satisfied in the optimal solution of the partial problem (with only a subset of the configurations) and therefore guaranteeing feasibility, we face the challenge of completing the maximum number of configurations of the iteration. After using the Multi-job Configuration Creator and analyzing its results, we came to the

3.5. Step 2: Multi-Start Configuration Generation Phase

47

Algorithm 8 Generate Multi-Job Configuration

```

function GENERATEMULTIJOBCONFIGURATION( $j_{base}$ )
   $\mathcal{T}_{req} \leftarrow j_{base}.requiredTools$ 
   $v \leftarrow capacity - |\mathcal{T}_{req}|$  ▷ Number of vacant slots
   $\mathcal{J}_{pending} \leftarrow jobs \setminus \{j_{base}\}$ 
  while  $v > 1$  and  $\mathcal{J}_{pending} \neq \emptyset$  do
     $\mathcal{J}_{sampled} \leftarrow \text{Shuffle}(\mathcal{J}_{pending})$ 
    for all  $j \in \mathcal{J}_{sampled}$  do
       $\mathcal{T}_{new} \leftarrow \mathcal{T}_{req} \cup j.requiredTools$ 
      if  $|\mathcal{T}_{new}| \leq capacity$  then
         $\mathcal{T}_{req} \leftarrow \mathcal{T}_{new}$ 
         $v \leftarrow capacity - |\mathcal{T}_{req}|$ 
         $\mathcal{J}_{pending} \leftarrow \mathcal{J}_{pending} \setminus \{j\}$ 
      else
         $\mathcal{J}_{pending} \leftarrow \mathcal{J}_{pending} \setminus \{j\}$ 
      end if
    end for
  end while
  while  $|\mathcal{T}_{req}| < capacity$  do ▷ Fill remaining slots
     $t_{next} \leftarrow \text{GETNEXTTOOL}(\mathcal{T}_{req})$ 
     $\mathcal{T}_{req} \leftarrow \mathcal{T}_{req} \cup \{t_{next}\}$ 
  end while
  return  $\mathcal{T}_{req}$ 
end function

```

conclusion that despite creating very good configurations, when used repeatedly many times with the same job as base, a problem comes up, which we explain with an example.

If we have an instance with four jobs i, j, k, l for which $T_i = \{1, 2\}, T_j = \{1, 3\}, T_k = \{4, 5\}, T_l = \{4, 6\}$, and a magazine capacity $b = 3$. It is easy to see that we have two pairs of jobs (i, j) and (k, l) for which there is one configuration that satisfy both jobs of the pair: $\{1, 2, 3\}$ for the pair (i, j) and $\{4, 5, 6\}$ for the pair (k, l) . We then note that both configurations have the same size as the magazine capacity, which means no other tool will be added to them. This generates an infinite loop: when starting the Multi-job Configuration Creator by either jobs i or j , we will always create configuration $\{1, 2, 3\}$. When starting the Multi-job Configuration Creator by either jobs k or l , on the other hand, we will always create configuration $\{4, 5, 6\}$.

This scenario motivated the creation of a second option for a configuration creator.

3.5.2.1 Single-job Configuration Creator

Our second idea, the Single-job Configuration Creator, has two main purposes: first, it aims to find the smooth transitions between clusters of similar jobs (in regards to their tool requirement sets), in some way complementing the Multi-job Configuration Creator. Second, the Multi-job Configuration Creator gets stuck in some specific cases of instances, creating the need of a more variable and even unpredictable configuration creator, like the Single-job Configuration Creator.

The idea for this configuration creator is much simpler: it also takes one specific job j as base and fixes its required tools in the configuration. Then it calls the Next Tool finder $b - |T_j|$ times until the configuration is complete. Of course, by using any specific Next Tool Strategy, it will be much more prone to follow this strategy's features.

The logic of the Single-job Configuration Creator is also shown in algorithm 9.

Algorithm 9 Generate Single-Job Configuration

```

function GENERATESINGLEJOBCONFIGURATION( $j_{base}$ )
   $\mathcal{T}_{req} \leftarrow j_{base}.requiredTools$ 

  while  $|\mathcal{T}_{req}| < capacity$  do           ▷ Fill remaining slots until capacity is reached
     $t_{next} \leftarrow \text{GETNEXTTOOL}(\mathcal{T}_{req})$ 
     $\mathcal{T}_{req} \leftarrow \mathcal{T}_{req} \cup \{t_{next}\}$ 
  end while

  return  $\mathcal{T}_{req}$ 
end function

```

3.5.2.2 High-level Configuration Creation

Now having both Multi-job and Single-job Configuration Creators, we are able to find the best configurations for both satisfying many jobs and smoothing transitions between very different configurations. In this section, we will explore how both methods are used.

Instead of choosing one specific configuration creator for each iteration of the multi-start, we instead prioritize one over the other. The most promising configuration creator is the Multi-job, as it maximizes our odds of finding configurations that can satisfy multiple

3.5. Step 2: Multi-Start Configuration Generation Phase

49

jobs at once, so for every next job chosen using the Next Job Strategy of the iteration, we give it one shot of finding a new configuration. If it fails to create a new configuration, the same base job is then used to try to create a new configuration by using the Single-job Configuration Creator.

The logic of the complete configuration creation (both the base and the extra configurations) is shown in algorithm 10.

Algorithm 10 Generate All Configurations

```

function GENERATEALLCONFIGURATIONS(amount_of_configs)
  config_index  $\leftarrow$  1
   $\triangleright$  Guarantee each job gets at least one configuration

  for each job  $\in$  jobs do
    is_new  $\leftarrow$  GENERATEMULTIJOBCONFIGURATION(job, config_index)
    if is_new then
      config_index  $\leftarrow$  config_index + 1
    end if
  end for

   $\triangleright$  Create extra configurations using next job strategy
  stall_count  $\leftarrow$  0
  while config_index  $\leq$  amount_of_configs and stall_count < stall_limit do
    selected_job  $\leftarrow$  GETNEXTJOB
    is_new  $\leftarrow$  GENERATEMULTIJOBCONFIGURATION(selected_job, config_index)

    if is_new then
      config_index  $\leftarrow$  config_index + 1
      stall_count  $\leftarrow$  0
    else
      is_new  $\leftarrow$  GENERATESINGLEJOBCONFIGURATION(selected_job, config_index)
      if is_new then
        config_index  $\leftarrow$  config_index + 1
        stall_count  $\leftarrow$  0
      else
        stall_count  $\leftarrow$  stall_count + 1
      end if
    end if
  end while
end function

```

The complete logic for any particular multi-start in regards to extra configuration creation is also shown in the flow diagram of figure 3.2.

3.5.3 Optimization

We then take all of the configurations generated by the logic described in the previous sections and use them as the graph for which we optimize and obtain an upper bound to the problem.

The optimization is handled by the Gurobi solver, using its python interface, Gurobipy. We add the objective function and all of the constraints except for the GSEC (Generalized Sub-tour Elimination Constraints) 2.6. As those are the constraints that grow exponentially when the number of configurations grows, we opted for using the *Params.LazyConstraints* = 1 option in Gurobipy. This means the solver will solve the problem without those constraints and add it on-the-go for each iteration, during the branch-and-cut process via a callback function.

The callback function was provided by us to the solver, checking for sub-routes in the current solution and returning which groups of configurations must be handled by adding their GSEC. By incorporating this logic, we implement a custom cutting-plane algorithm where violated GSECs are added as cutting planes during the branch-and-cut process, effectively tightening the search space only where necessary and making it much less computationally expensive.

3.5.4 KTNS improvement

After obtaining a feasible solution and therefore an upper bound to the problem in the optimization step, we aim to improve the current solution by applying the KTNS policy, exploring its feature of being polynomial in time.

The core idea of this step is to explore the feature of the obtained feasible solution of the optimization phase as a sequence of configurations, which can easily be converted to many different sequences of jobs. In the worst case scenario, no configuration of the feasible solution satisfied more than one job, and hence we have exactly one job sequence to derive from it. For most cases, however, there will be configurations capable of satisfying more than one job, and hence it creates more than one job sequence.

Another feature that we explore is the fact that one job might be satisfied by more than one configuration selected in the feasible solution. This means we can place that specific job in different parts of the job sequence.

As the amount of different job sequences created is unpredictable and also has an exponential factor to it (as one configuration that satisfied multiple jobs will generate multiple sub-sequences), we also define a hard limit in order to keep this phase efficient and not very costly.

The logic of the KTNS improvement phase is shown in algorithm 11.

3.6 Step 3: Mixing and Improving Configurations Phase

After generating multiple feasible solutions to the problem, we then have the opportunity to profit on their best features in order to gather a pool of the best configurations and therefore generate an even better solutions. That is why we have created a mixing and improving configurations phase.

3.6.1 Selection of configurations from upper bound solutions in multi-start phase

We start this step of the heuristic with a large number of configurations that were created in previous steps. The selection of a subset of this configurations is a key step in order to mix their good features and find better solutions.

This step also includes two sub-steps that are similar goal-wise to those of the Multi-Start phase: first, we must ensure feasibility of the problem by selecting at least one configuration that satisfies the tool requirements of each job in the instance. Then, we are free to fill the configurations set with other configurations based on some criteria.

Algorithm 11 Phase II Improvement by KTNS with Bounded Combinations

```

function IMPROVEBYKTNS( $\mathcal{C}_{seq}$ , capacity,  $\mathcal{T}_{req}$ ,  $\mathcal{J}_{sat}$ ,  $B_{best}$ )
   $B_{current} \leftarrow B_{best}$ 
   $\mathcal{C}_{opt} \leftarrow \emptyset$ 
  for  $i \leftarrow 1$  to  $|\mathcal{C}_{seq}|$  do                                      $\triangleright$  Multi-start over rotations
     $\mathcal{C}_{rot} \leftarrow \text{Rotate}(\mathcal{C}_{seq}, i)$ 
     $\mathcal{P} \leftarrow \emptyset$                                             $\triangleright$  Permutation sets per configuration
    for all  $c \in \mathcal{C}_{rot}$  do
       $\mathcal{J}_c \leftarrow \mathcal{J}_{sat}[c]$ 
       $\mathcal{P}_c \leftarrow \text{RANDOMPERMUTATIONS}(\mathcal{J}_c)$ 
       $\mathcal{P} \leftarrow \mathcal{P} \cup \{\mathcal{P}_c\}$ 
    end for
     $n \leftarrow 0$ 
     $\mathcal{S}_{seen} \leftarrow \emptyset$ 
    for all  $\pi \in \text{CARTESIANPRODUCT}(\mathcal{P})$  do
       $\mathcal{J}_{seq} \leftarrow \emptyset$ 
       $\mathcal{J}_{used} \leftarrow \emptyset$ 
      for all  $block \in \pi$  do
        for all  $j \in block$  do
          if  $j \notin \mathcal{J}_{used}$  then
             $\mathcal{J}_{used} \leftarrow \mathcal{J}_{used} \cup \{j\}$ 
            Append  $j$  to  $\mathcal{J}_{seq}$ 
          end if
        end for
      end for
      if  $\mathcal{J}_{seq} \notin \mathcal{S}_{seen}$  then
         $\mathcal{S}_{seen} \leftarrow \mathcal{S}_{seen} \cup \{\mathcal{J}_{seq}\}$ 
         $(B_{new}, \mathcal{C}_{new}) \leftarrow \text{APPLYKTNS}(\mathcal{J}_{seq}, \text{capacity}, \mathcal{T}_{req})$ 
        if  $B_{new} < B_{current}$  then
           $B_{current} \leftarrow B_{new}$ 
           $\mathcal{C}_{opt} \leftarrow \mathcal{C}_{new}$ 
        end if
      end if
       $n \leftarrow n + 1$ 
      if  $n \geq N_{max}$  then                                        $\triangleright$  Global limit on tested sequences
        break
      end if
    end for
  end for
  return  $(B_{current}, \mathcal{C}_{opt})$ 
end function

```

3.6. Step 3: Mixing and Improving Configurations Phase

53

For this step, we already have likely highly useful configurations, as they were the configurations used in the upper bound solution of the problems solved previously. So we first get for each job the pool of configurations that satisfy their tool requirements and select randomly one configuration, to ensure feasibility.

Following that, we use another weighted-random selection from the configurations left unselected: the weights used are the amount of different jobs that each configuration satisfies. Using this selection we complete the configuration pool and are ready to optimize.

The logic for the selection of configurations is shown in algorithm [12](#).

3.6.2 Optimization and KTNS improvement

The optimization and the KTNS improvement steps are done in the same manner as the previous steps.

3.6. Step 3: Mixing and Improving Configurations Phase

54

Algorithm 12 Configuration Selection For Mixing and Improving Phase

```

function CONFIGURATIONSELECTION( $I, O, M_{max}$ )
   $\mathcal{R} \leftarrow \text{heuristicRuns}$ 
   $\mathcal{C}_{all} \leftarrow \emptyset$ 
   $k \leftarrow 1$ 
   $\triangleright$  Collect all distinct configurations from previous runs

  for all  $r \in \mathcal{R}$  do
    for all  $C \in r.\text{optimalConfigs}$  do
      if  $C \notin \mathcal{C}_{all}$  then
         $\mathcal{C}_{all}[k] \leftarrow C$ 
         $k \leftarrow k + 1$ 
      end if
    end for
  end for
   $\triangleright$  Update satisfying configurations per job

   $\mathcal{S}_{job} \leftarrow \emptyset$ 
  for all  $j \in \text{jobs}$  do
     $\mathcal{S}_{job}[j] \leftarrow \emptyset$ 
    for all  $(c, \mathcal{T}_c) \in \mathcal{C}_{all}$  do
      if  $j.\text{requiredTools} \subseteq \mathcal{T}_c$  then
         $\mathcal{S}_{job}[j] \leftarrow \mathcal{S}_{job}[j] \cup \{c\}$ 
      end if
    end for
  end for
   $\triangleright$  Random selection with coverage guarantee

  if  $|\mathcal{C}_{all}| > M_{max}$  then
     $\mathcal{C}_{sel} \leftarrow \emptyset$ 
     $\triangleright$  Ensure every job is satisfied by selecting one of its configurations
    for all  $j \in \text{jobs}$  do
       $c \leftarrow \text{RANDOMCHOICE}(\mathcal{S}_{job}[j])$ 
       $\mathcal{C}_{sel} \leftarrow \mathcal{C}_{sel} \cup \{c\}$ 
    end for
     $\triangleright$  Compute weights based on number of satisfied jobs

     $\mathcal{W} \leftarrow \emptyset$ 
    for all  $c \in \mathcal{C}_{all}$  do
      if  $c \notin \mathcal{C}_{sel}$  then
         $w_c \leftarrow |\{j \in \text{jobs} \mid c \in \mathcal{S}_{job}[j]\}|$ 
         $\mathcal{W}[c] \leftarrow w_c$ 
      end if
    end for
     $\triangleright$  Weighted random completion until reaching the limit

     $\mathcal{C}_{rem} \leftarrow \mathcal{C}_{all} \setminus \mathcal{C}_{sel}$ 
    if  $|\mathcal{C}_{sel}| < M_{max}$  and  $\mathcal{C}_{rem} \neq \emptyset$  then
       $\mathcal{C}_{rnd} \leftarrow \text{WEIGHTEDRANDOMSAMPLE}(\mathcal{C}_{rem}, \mathcal{W}, M_{max} - |\mathcal{C}_{sel}|)$ 
       $\mathcal{C}_{sel} \leftarrow \mathcal{C}_{sel} \cup \mathcal{C}_{rnd}$ 
    end if
  else
     $\mathcal{C}_{sel} \leftarrow \mathcal{C}_{all}$ 
  end if
  return  $\mathcal{C}_{sel}$ 
end function

```

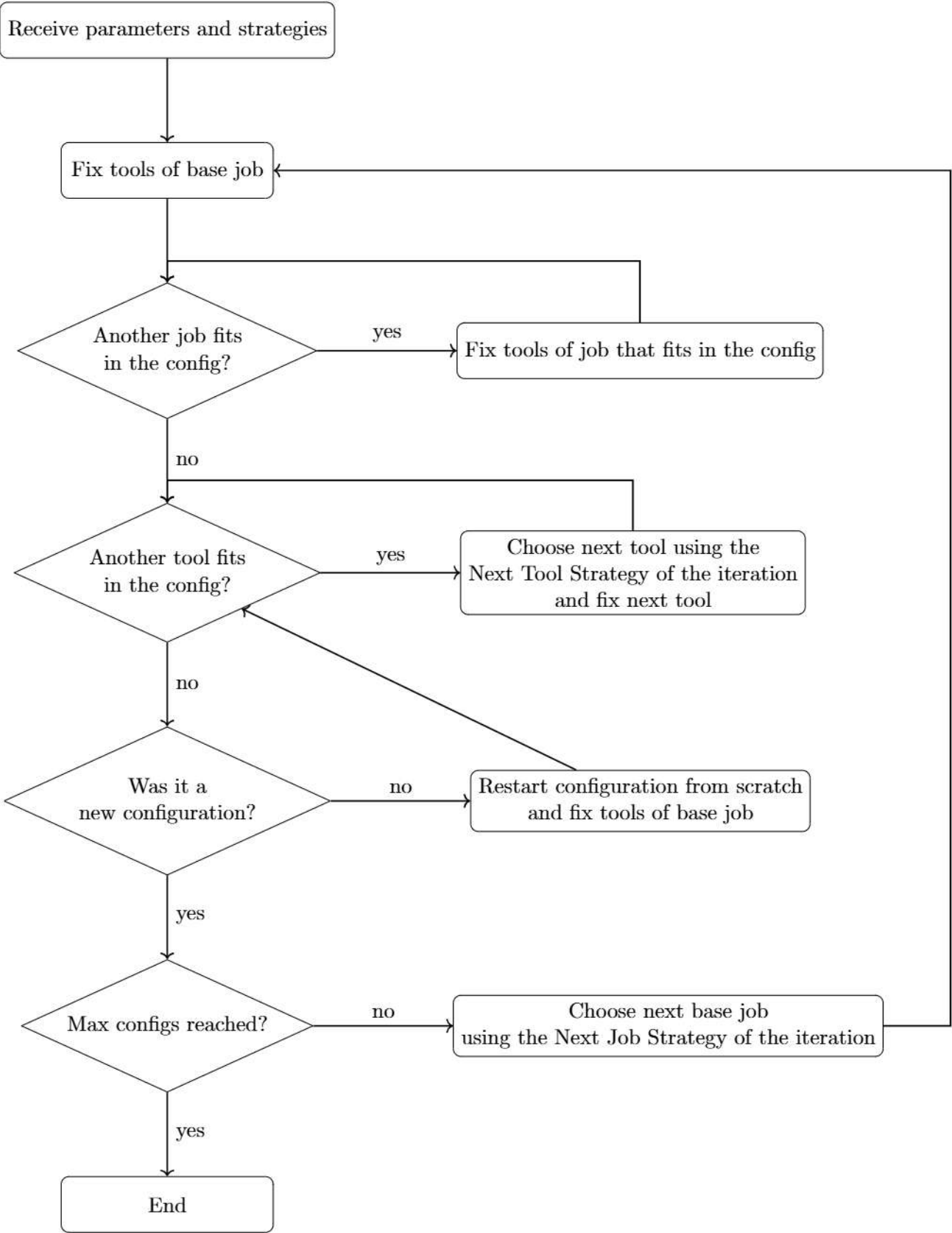


Figure 3.2: Flow diagram describing the extra configuration creation process of a multi-start run in the heuristic method.

Chapter 4

Results and discussion

For the tests of this work, we have used three large groups of instances:

1. 160 instances from [4] divided into 4 groups of 40 instances: datA, datB, datC and datD;
2. 160 instances from [8] divided into 4 groups of 40 instances: C1, C2, C3 and C4;
3. 1350 instances from [21] divided into 5 groups of varying sizes: GroupA, GroupB, GroupC, GroupD and GroupE.

Those are the three classical benchmark groups of instances that are widely used by authors in the SSP field. The proposed heuristic method was ran for every instance with a time limit of 5200 seconds for each instance. All tests were ran in a [ADD MACHINE DETAILS].

This results section is divided into three parts. First, we will compare the obtained upper bound with the heuristic method with some instances of group datB from [4], as they are medium-sized instances for which the exact models from section were able to find the optimal solution.

Then, we will compare the results for all three groups of instances to the best heuristic results as far we were able to find, from [14]. In this comparison, we will keep the common way of displaying SSP results as seen in both [15] and [14]: we will further group the instances of each of the three datasets into smaller subgroups by a tuple (number of jobs, number of tools, capacity). For each of those subgroups, we calculate the average of the best solution found by the heuristic in each of the instances of the subgroup, and the average time spent to find those solutions.

Finally, we will present some results regarding the internal comparison for our proposed heuristic between the different parameters and strategies that were tested. The idea is to understand which were the best performing variations and hence provide valuable insight for future work. In this section, we will only use the instances from [4], as they have very

organized features: groups datA, datB, datC and datD are clearly separated in clusters by number of jobs and number of tools. Inside each group, there are yet another organized subgroups, (e.g. datA1, datA2, datA3, datA4), for which the size of the set of jobs and the size of the set of tools is usually the same. The magazine capacity, on the other hand, increases for each of the subgroups. This is really helpful in comparisons as it is more reliable to gather instances from the same subgroup and see the differences between the different approaches from the multi-start iterations.

4.1 General results of the heuristic method

As the instances grow, even the exact formulations are not able to find the optimal value for many instances. For cases where at least one of the exact formulations found the optimal value and hence the optimal is known, we will compare the heuristic's obtained upper bound with the optimal value, of course.

4.1.1 General results for medium-sized instances with known optimal solution

For some of the small and medium instances, it may be interesting to analyze the obtained values with the heuristic and compare it to the optimal values obtained by exact models, when one or both models found the optimal solution. For group datA from [4], the heuristic was able to find the optimal solution for all of the 40 instances. Table 3 shows for both models and the heuristic if the optimal value was found as the run time in seconds for group datB.

The results show that the heuristic obtained good results in the group of instances datB for which the optimal value is known. The heuristic was able to find the optimal solution for the all 40 datA instances and for 23 out of 28 datB instances for which the optimal solution is known, a total of 93% of times. For the other 5 instances, the maximum gap in switches from the obtained upper bound and the optimal solution was 2. This indicates that the heuristic is largely capable of finding the configurations that could be used in an optimal solution.

4.1. General results of the heuristic method

58

Instance	SSPMF		CGL		Optimal Value	Heuristic	
	Opt?	Time	Opt?	Time		UB	Time
B1-10	Yes	1616	Yes	797	22	23	80
B1-2	No	5200	Yes	81	26	26	168
B1-3	No	5200	Yes	442	23	23	146
B1-4	Yes	1177	No	5200	18	20	619
B1-6	Yes	2487	Yes	292	21	23	231
B1-7	Yes	3972	No	5200	15	15	910
B1-8	No	5200	Yes	636	22	22	706
B1-9	Yes	2712	No	5200	19	19	231
B2-7	Yes	0	No	5200	10	10	523
B3-1	Yes	0	No	5200	10	10	827
B3-10	Yes	12	No	5200	10	11	425
B3-3	Yes	759	No	5200	10	11	349
B3-4	Yes	2	No	5200	8	8	544
B3-5	Yes	842	No	5200	11	11	490
B3-6	Yes	425	No	5200	10	10	219
B3-7	Yes	0	No	5200	8	8	325
B3-8	Yes	3	No	5200	9	9	150
B3-9	Yes	3	No	5200	9	9	402
B4-1	Yes	0	No	5200	8	8	439
B4-10	Yes	0	No	5200	8	8	284
B4-2	Yes	1	No	5200	8	8	214
B4-3	Yes	0	No	5200	7	7	251
B4-4	Yes	0	No	5200	6	6	189
B4-5	Yes	0	No	5200	8	8	331
B4-6	Yes	0	No	5200	7	7	311
B4-7	Yes	0	No	5200	6	6	262
B4-8	Yes	1	No	5200	7	7	199
B4-9	Yes	0	No	5200	7	7	309

Table 3: Optimal solution, obtained upper bound with heuristic and computational times for heuristic and SSPMF and CGL models for datB instances for which optimal was found with exact models.

4.1.2 General results of the heuristic method for all instances from [4]

We then explore the results obtained by the heuristic itself to better understand how well it performed considering some metrics. As explained previously, this section will only analyze the results from instances of [4], as they provide much clearer insight of the heuristic performance. On top of that, to improve readability and considering that datA instances are trivial, we will only include datB, datC and datD instances in the remaining results of this section. The only exception made is for one graph where we will

analyze instances for which the optimal value was found using the exact models, hence datA instances are relevant for this case.

One of the main issues to be concerned when using a multi-start strategy is to understand if the obtained solutions for each iteration are uniformly good in some way (as computing one totally bad solution will waste heuristic time without helping in getting the best results). In figure 4.1 we show the distribution for each instance of their best to worst solution gap (considering all 12 multi-start runs and also the improvement phase final solution) by the standard deviation for the obtained upper bounds in those solutions. Each instance is then represented by a point in graph and we add colors to help identify the groups of instances.

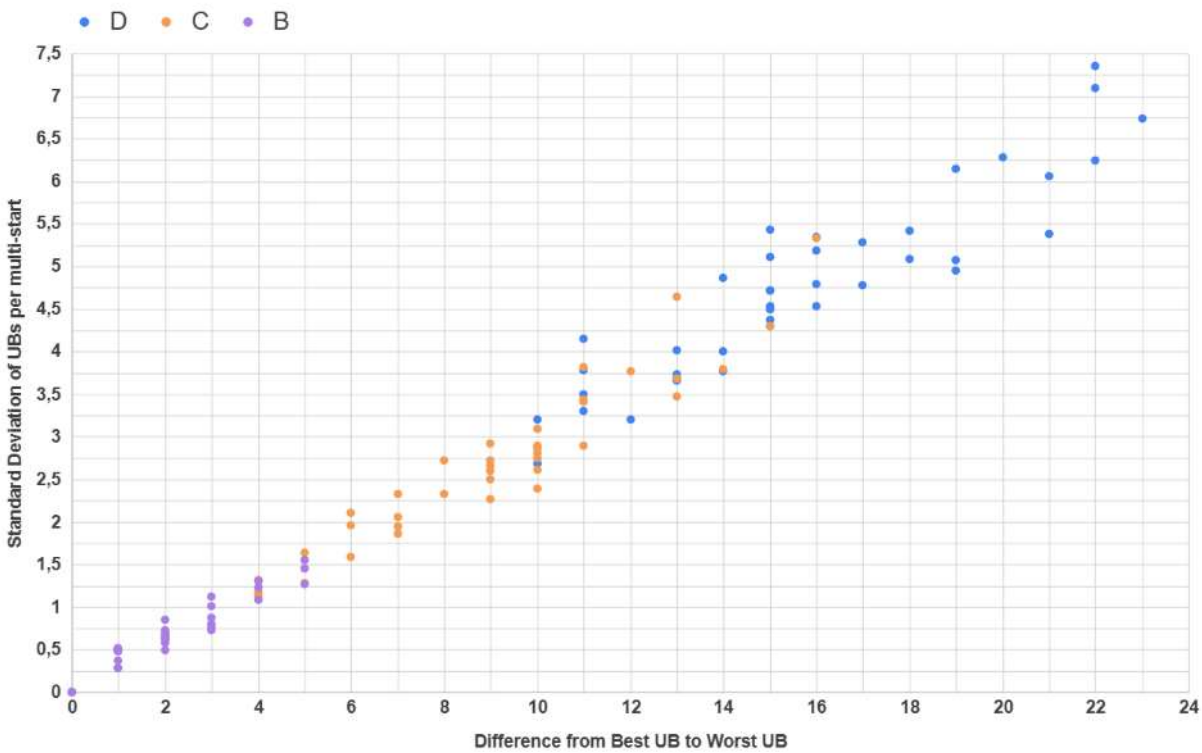


Figure 4.1: Distribution of instances for best to worst solution gap by standard deviation of obtained solutions.

We can see in the graph that as the instances grow (from datB to datC to datD), the difference in switches from the best solution found to the worst solution found increases, which is expected as the optimal value too increases. However, we were able to keep all instances from datB within a range of 2 to 5 switches of difference and most instances of datC within a range of 4 to 10 switches of difference, which shows good consistency between every heuristic run of the same instance. We can also notice that larger instances such as these in datD group generate a much larger difference between runs and also they are further apart from each other.

4.1.2.1 Multi-start results before applying KTNS

In a general manner, to evaluate how well the heuristic base is constructed, we can verify, for known optimal instances, how many times we were able to find the optimal solution in some of the multi-start runs. Figure 4.2 shows the percentage of multi-start runs that were able to find the optimal solution even before of applying the KTNS policy, for each instance.

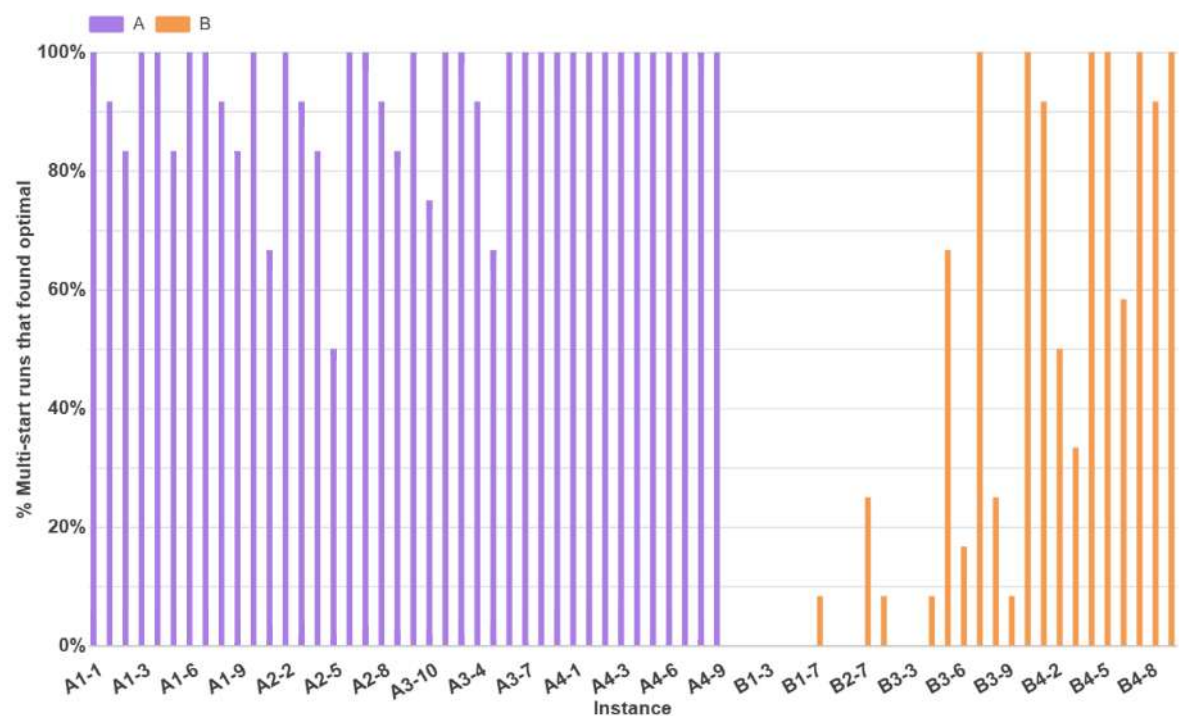


Figure 4.2: Percentage of multi-start runs that found the known optimal solution for each instance.

The graph shows that for every instance in group datA we were able to find the optimal solution in at least 50% of the multi-start runs, and in many cases every single multi-start run found the optimal solution. For instances in group datB, in some cases we were also able to find the optimal solution in all multi-start runs, but it was clearly less frequent. For most instances, however, at least one multi-start run was able to find the optimal solution even before applying the KTNS policy.

4.1.2.2 KTNS improvement results

Applying the KTNS policy at the end of each multi-start iteration has proven to be a reliable, cheap way to improve the obtained solution by the heuristic method. In figure 4.3 we show the distribution for each instance of the average improvement we obtained simply by applying the KTNS policy at each multi-start and also at the end of the improvement phase by the average improvement. Each instance is then represented by a point in graph and we add colors to help identify the groups of instances.

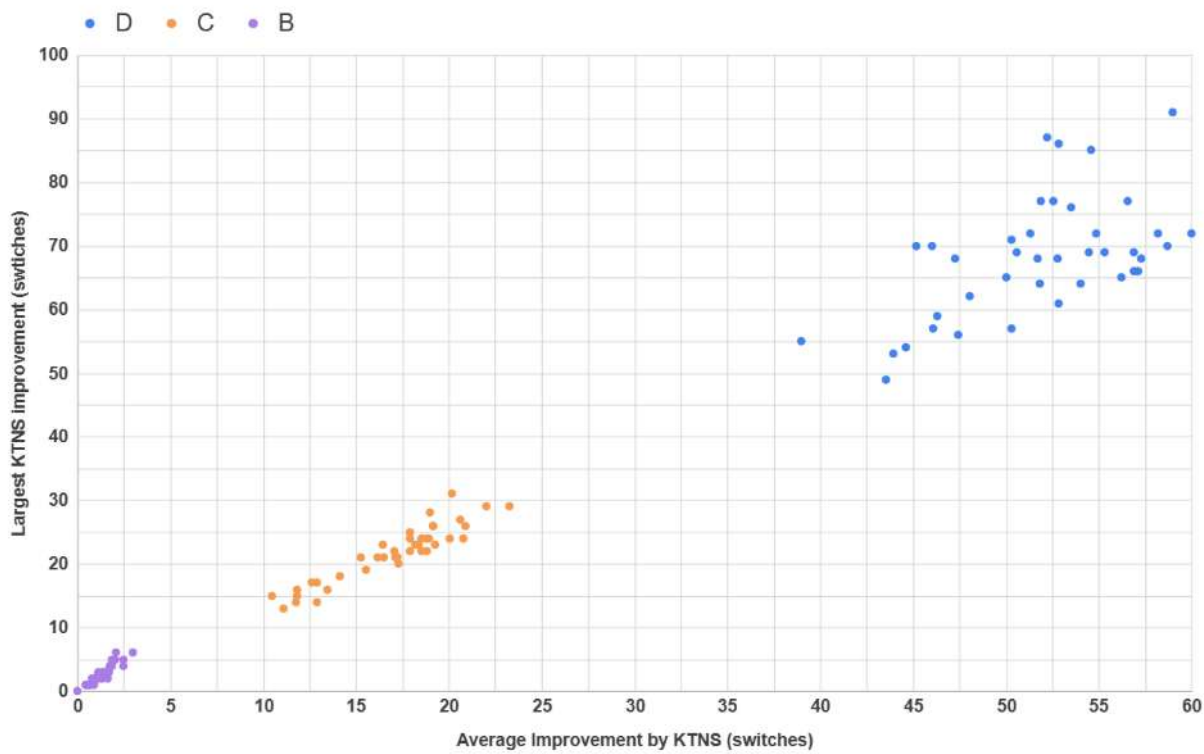


Figure 4.3: Distribution of instances for Average Improvement by applying KTNS by Largest Improvement by applying KTNS, both in number of switches reduced.

The graph shows clearly the impact of applying the KTNS policy at the end of every multi-start run, specially for instances for which we did not have a large percentage of multi-starts that already found the optimal solution. For example, the best KTNS improvement in switches for group datB was within the range from 1 to 6. And the highest average improvement with KTNS was about 3 switches. When we look at group datC, its application is even more effective as we are able to reduce up to 23 switches on average and up to 31 switches at best. Nevertheless, the most impressive results for the KTNS policy are seen in larger instances, from group datD: the best improvement for each instance was in the range from 49 up to 91 switches, and we had instances for which the average improvement was 60 switches, proving its efficacy, specially considering that

it is polynomial in time.

4.1.3 Mixing and Improvement phase results

After analyzing the results of the multi-start iterations, it is important to understand if the final step in our heuristic actually improves the obtained results from the previous 12 runs of the multi-start. In this case, the idea is to compare the best obtained upper bound from the multi-start runs (which is until the mixing and improvement phase the current best solution) with two solutions: first, the obtained upper bound in the optimization step of the mixing and improvement phase. And second, the obtained upper bound after applying the KTNS policy at the end of the mixing and improvement phase. This way, we are able to understand how effective is this last phase for each instance.

Considering the 120 instances from groups datB, datC and datD, the exact models were able to find the optimal solution for 19 of them. This means that the other 101 instances for which the exact models were not able to find the optimal solution are possible that the mixing and improvement solution will find a better solution than the current one. Out of those 101 instances, the mixing and improvement solution actually improved the current solution in 27 of them. Table 4 shows, for those 27 instances, what the current bound was (the best multi-start upper bound after applying the KTNS policy), the solution found by optimizing the mixed configurations before applying the KTNS policy and after applying the KTNS policy. Finally, we show the improvement in switches obtained by the mixing and improvement phase.

The results show that in only 7 instances the mixing and improvement phase was able to improve the solution before the KTNS policy. For those 7 instances, the KTNS was not able to further improve the solution. This indicates a great opportunity to enhance the heuristic logic in this last step in order to obtain better results. Here we state some possibilities of next steps for the mixing and improvement phase:

- Increasing both the maximum configurations allowed and the maximum computational time for this step only: for many cases, the maximum of 100 configurations was reached by this step, hence it could be interesting to allow more time for it in order to find better results;
- Changing the criteria to select the configurations from the pool of configurations used by the upper bound solutions of the multi-start runs: for this heuristic, we

4.1. General results of the heuristic method

63

Instance	Multi-Start	Mixing and Improving Phase		
	UB post-KTNS	UB pre-KTNS	UB post-KTNS	Improvement
B1-10	24	23	23	1
B1-2	27	26	26	1
B1-3	24	23	23	1
B1-8	23	22	22	1
B1-9	20	19	19	1
B2-8	14	13	13	1
B2-9	14	13	13	1
B3-10	12	12	11	1
C1-1	94	104	93	1
C1-10	81	97	79	2
C1-4	91	104	90	1
C2-1	78	97	77	1
C2-10	65	77	64	1
C2-2	83	104	82	1
C2-8	78	95	76	2
C3-1	55	75	53	2
C3-7	56	70	51	5
C3-8	55	76	53	2
C3-9	59	79	57	2
C4-3	32	42	31	1
C4-4	31	39	30	1
C4-9	36	46	34	2
D1-9	212	248	210	2
D2-1	168	221	166	2
D2-6	200	255	199	1
D2-9	182	238	178	4
D3-10	154	228	151	3

Table 4: Best upper bound obtained by a multi-start, upper bound obtained before and after the KTNS policy for the mixing and improving phase and improvement from that phase (in switches) for instances in which the mixing and improving phase was able to improve the solution.

have proposed a biased-random way of selecting the configurations from the pool, but adding some rational to it might improve the ability of this phase of getting the best features from each multi-start iteration;

- Start using the configurations generated by the KTNS policy of each multi-start run: the proposed mixing and improvement phase uses the configurations obtained by each multi-start run before applying the KTNS policy. This work, however, showed great improvements obtained by this policy and hence a next step could be to get the KTNS solution and translate it into magazine configurations that could be used in this last step of the heuristic method.

4.2 Comparison with the heuristics from [15] and [14] for all instances

The results obtained and reported by [14] using heuristic methods to solve this variation of the SSP is the best that we were able to find. They reported better results than those on [15] by using a hybrid genetic search. They did not provide as results the obtained upper bound for every instance of the three subgroups separately (as it would account for more than 1500 instances), instead they used the same summary method as [15] and provided the average of the obtained upper bounds for each of the subgroups we described, grouping by the same number of jobs, number of tools and capacity.

Hence for this section we will compare those two heuristics which are recognized as having good results for the SSP with the new proposed heuristic. The Iterated Local Search from [15] will be referred as ILS, the Hybrid Genetic Search from [14] will be referred as HGS, and finally our proposed heuristic will be referred as New.

The comparisons will follow the pattern of results seen in both [15] and [14], for which we take the averages for the runs for each instance of a subgroup (where a subgroup is a group of instances with same number of jobs, same number of tools and same magazine capacity). For each subgroup, we will take the average of the best upper bound obtained for each instance, S^* , and the average time spent solving each instance of the subgroup, T . Those two metrics will be displayed for all three heuristic methods.

It is important to notice that in this section exclusively we will consider that the first machine setup counts as switches. This basically means that we will add the magazine capacity to the obtained upper bound in both our work and in [14], while [15] already did that in their paper. We do this only to compare equally the results between studies, as there is no practical difference for those methods.

4.2.1 Results for Catanzaro's instances from [4]

The dataset from [4] has 160 instances, divided into four groups of 40 instances for which the number of jobs and number of tools is the same inside the group. Those groups

4.2. Comparison with the heuristics from [15] and [14] for all instances

65

instance group	n	m	b	i	ILS		HGS		New	
					S*	T	S*	T	S*	T
datA1	10	10	4	10	12.5	0.09	12.5	0.07	12.5	28.3
datA2	10	10	5	10	10.8	0.09	10.8	0.06	10.8	55.0
datA3	10	10	6	10	10.1	0.05	10.1	0.06	10.1	52.0
datA4	10	10	7	10	10	0.04	10.0	0.05	10.0	43.5
datB1	15	20	6	10	26.5	1.09	26.5	0.88	27.1	418.5
datB2	15	20	8	10	21.7	1.38	21.7	0.79	22.3	896.7
datB3	15	20	10	10	19.7	1.121	19.7	0.69	19.9	407.3
datB4	15	20	12	10	19.2	0.714	19.2	0.6	19.2	278.9
datC1	30	40	15	10	98.9	99.471	98.5	74.71	108.3	954.3
datC2	30	40	17	10	82.5	137.084	82.4	71.77	91.9	957.4
datC3	30	40	20	10	66.6	172.586	66.6	60.58	73.7	1332.1
datC4	30	40	25	10	51.3	137.789	51.2	48.01	56.8	731.7
datD1	40	60	20	10	198	488.661	196.5	501.21	219.2	419.9
datD2	40	60	22	10	173.6	706.126	172.3	437.85	194.4	389.8
datD3	40	60	25	10	146.2	1069.98	145.3	466.12	167.8	938.2
datD4	40	60	30	10	115.2	1518.8	114.3	484.14	134.0	1394.4

Table 5: Obtained upper bounds with ILS, HGS and New Heuristics and time in seconds to obtain the upper bound for every instance from Catanzaro dataset [4].

are further subdivided into subgroups of 10 instances each for which the capacity increases at each subgroup.

Table 5 shows the comparative results between the three heuristic methods for all 16 subgroups of instances in this dataset.

We can observe from the results that the HGS has obtained the best upper bound for all 16 subgroups for this dataset, obtaining the best results. The ILS heuristic was able to match this best values for 9 of the subgroups, however its computational time was much greater than those from the HGS. The New heuristic was able to match the best values for 5 of the subgroups, including 4 from the most trivial group, datA. For some instances of group datD, the New heuristic took less time than the other heuristics.

4.2.2 Comparison with [14] for Crama's instances from [8]

The dataset from [8] also has 160 instances, divided into four groups of 40 instances for which the number of jobs and number of tools is the same inside the group. Those groups are further subdivided into subgroups of 10 instances each for which the capacity increases at each subgroup.

4.2. Comparison with the heuristics from [15] and [14] for all instances

66

n	m	b	i	ILS		HGS		New	
				S*	T	S*	T	S*	T
10	10	4	10	13.1	0.08	13.1	0.08	13.1	16.9
10	10	5	10	11.2	0.08	11.2	0.07	11.2	42.8
10	10	6	10	10.3	0.06	10.3	0.06	10.3	45.3
10	10	7	10	10.1	0.04	10.1	0.06	10.1	22.5
15	20	6	10	26.6	0.84	26.6	0.93	26.9	407.5
15	20	8	10	21.7	1.09	21.7	0.82	21.9	739.9
15	20	10	10	20.1	0.87	20.1	0.73	20.1	489.3
15	20	12	10	19.6	0.6	19.6	0.64	19.6	336.5
30	40	15	10	106.4	86.81	106.1	75.85	114.2	961.2
30	40	17	10	88.3	132.01	88.2	66.94	97.8	825.1
30	40	20	10	70.4	173.47	70.2	64.4	78.5	1005.2
30	40	25	10	52.9	146.81	52.8	62.61	59.2	797.3
40	60	20	10	198.4	441.66	197.2	512.09	218.1	388.8
40	60	22	10	173.5	665.13	172.5	490.88	196	493.1
40	60	25	10	146	1016.37	145.2	478.33	166.8	769.6
40	60	30	10	113.9	1484.97	113.2	408.9	133.8	1559.8

Table 6: Obtained upper bounds with ILS, HGS and New Heuristics and time in seconds to obtain the upper bound for every instance from Crama dataset [8]

Table 6 shows the comparative results between the three heuristic methods for all 16 subgroups of instances in this dataset.

We can observe that all three heuristics were able to find the same solution for the most trivial instances from the first 4 subgroups. This also happens for 2 out of 4 subgroups from the second group (with 15 jobs and 20 tools). However, the time used by ILS and HGS is much shorter to achieve those results. As instances grow, however, the results obtained by ILS and HGS are better than those from our heuristic method, and times spent trying to solve the instances are closer between methods.

4.2.3 Comparison with [14] for Yanasse's instances from [21]

The dataset from [21] has 1350 instances, divided into five groups of instances: GroupA, GroupB, GroupC, GroupD and GroupE, with 340, 330, 340, 260 and 80 instances, respectively. The number of jobs and the number of tools and the capacity vary inside each group and the number of instances of each tuple (n, m, b) is also different for each case.

Tables 7, 8, 9, 10 and 11 show the comparative results between the three heuristic methods for all 1350 instances in this dataset, representing groups GroupA, GroupB,

4.2. Comparison with the heuristics from [15] and [14] for all instances

67

n	m	b	i	ILS		HGS		New	
				S*	T	S*	T	S*	T
8	15	5	10	17	0.04	17	0.04	17	72.50
8	15	10	30	16.83	0.07	16.83	0.04	16.83	148.93
8	20	5	10	21.8	0.06	21.8	0.06	21.9	786.5
8	20	10	30	23.07	0.13	23.07	0.06	23.17	641.77
8	20	15	60	22.08	0.1	22.08	0.05	22.1	243.42
8	25	5	10	25.1	0.03	25.1	0.07	25.1	1640.2
8	25	10	30	28.2	0.12	28.2	0.08	28.3	1557.07
8	25	15	60	27.95	0.14	27.95	0.08	28.03	867.08
8	25	20	100	26.61	0.11	26.61	0.06	26.7	227.29

Table 7: Obtained upper bounds with ILS, HGS and New Heuristics and time in seconds to obtain the upper bound for GroupA from Yanasse dataset

n	m	b	i	ILS		HGS		New	
				S*	T	S*	T	S*	T
9	15	5	10	17.2	0.07	17.2	0.07	17.2	111.2
9	15	10	30	17.37	0.11	17.37	0.07	17.37	187.37
9	20	5	10	22.4	0.08	22.4	0.09	22.4	463.2
9	20	10	30	24.17	0.19	24.17	0.1	24.3	1050.77
9	20	15	60	22.6	0.18	22.6	0.08	22.68	217.45
9	25	5	10	25.4	0.05	25.4	0.11	25.4	1325.1
9	25	10	30	28.77	0.21	28.77	0.13	28.97	1809.73
9	25	15	50	29.74	0.33	29.74	0.13	29.9	1263.52
9	25	20	100	27.19	0.2	27.19	0.1	27.25	229.27

Table 8: Obtained upper bounds with ILS, HGS and New Heuristics and time in seconds to obtain the upper bound for GroupB from Yanasse dataset

GroupC, GroupD and GroupE, respectively.

The results confirm the trend seen in previous results. For some smaller instances, such as those in GroupA, our heuristic method is able to match the obtained upper bounds from both ILS and HGS, but needs much more time to do so. As instances grow, ILS and HGS grow their computational time, but are able to find better results than our heuristic. They tied or outperformed the new proposed heuristic for every subgroup of instances. The distance from their result to ours was smaller in this subset than for other subsets, which could be explained by also smaller upper bound values overall.

4.2. Comparison with the heuristics from [15] and [14] for all instances

68

n	m	b	i	ILS		HGS		New	
				S*	T	S*	T	S*	T
15	15	5	10	21.6	0.53	21.6	0.71	21.7	89.7
15	15	10	30	19.8	0.89	19.8	0.57	19.87	169.53
15	20	5	10	25.6	0.67	25.6	0.83	25.8	448.8
15	20	10	30	28.33	1.59	28.33	0.96	25.43	1049.2
15	20	15	60	25.52	1.71	25.52	0.76	25.82	315.88
15	25	5	10	32.5	0.65	32.5	1.02	32.6	419
15	25	10	30	35.07	1.92	35.07	1.28	35.63	1571.1
15	25	15	60	34.07	3.02	34.07	1.18	34.9	896.17
15	25	20	100	29.66	2.06	29.66	0.9	29.97	240.49

Table 9: Obtained upper bounds with ILS, HGS and New Heuristics and time in seconds to obtain the upper bound for GroupC from Yanasse dataset

n	m	b	i	ILS		HGS		New	
				S*	T	S*	T	S*	T
20	15	5	10	25.9	1.39	25.9	2.59	26.1	125
20	15	10	20	18.2	2.2	18.2	1.83	18.35	234.1
20	20	5	10	29.2	1.85	29.2	3.23	29.6	431.6
20	20	10	10	20.6	2.88	20.6	2.52	20.8	547.9
20	20	15	30	21.67	3.32	21.67	2.27	21.8	328.23
20	25	5	10	35.1	2.1	35.1	3.99	35.8	595.8
20	25	10	10	25.4	3.73	25.4	3.37	26	1336.1
20	25	15	40	36.25	7.41	36.25	3.88	37.25	644.75
20	25	20	40	26.15	3.45	26.15	2.69	26.48	359.78
25	15	10	10	15.9	3.97	15.9	3.78	16	230.8
25	20	10	10	21.6	9.73	21.6	6.58	22.8	553.9
25	20	15	10	22.6	9.93	22.6	6.74	23	243.4
25	25	10	10	26.6	11.68	26.6	8.6	28	992.7
25	25	15	10	25	7.83	25	6.21	25	289.5
25	25	20	30	25.5	6.89	25.5	5.97	25.77	255.5

Table 10: Obtained upper bounds with ILS, HGS and New Heuristics and time in seconds to obtain the upper bound for GroupD from Yanasse dataset

n	m	b	i	ILS		HGS		New	
				S*	T	S*	T	S*	T
10	10	4	10	13.5	0.08	13.5	0.07	13.5	9.5
10	10	5	10	11.2	0.08	11.2	0.06	11.2	34.6
10	10	6	10	10.3	0.05	10.3	0.06	10.3	52.5
10	10	7	10	10	0.03	10	0.06	10	26.3
15	20	6	10	27.4	0.77	27.4	0.92	27.8	406.1
15	20	8	10	22.2	0.9	22.2	0.84	22.7	1172.2
15	20	10	10	20.3	0.69	20.3	0.72	25.43	1049.2
15	20	12	10	20.2	0.62	20.2	0.69	20.2	433.4

Table 11: Obtained upper bounds with ILS, HGS and New Heuristics and time in seconds to obtain the upper bound for GroupE from Yanasse dataset

4.3 Comparison between multi-start strategies

Apart from analyzing generally how well the heuristic did perform in comparison with the two exact formulations, this work also aims to understand the proposed heuristic main strengths and flaws, in order to help future work based on this method to focus on what we could learn so far about it.

The definition of the parameters for each iteration of the heuristic method is key, as it has great influence in which configurations will be created and provided to the formulation and hence directly influence the obtained results. In particular, some of the parameters such as the next job strategy will highly define how well we explore or exploit the universe of possible configurations, as for many instances fixing the required tools of one specific job in the machine configuration will take most of the magazine capacity at once, for example.

For each of the three main variable parameters provided to the model, we will explore its isolate effect by considering the final result of each iteration that used each specific value of each parameter. For each parameter, we will explore the obtained upper bound values of each multi-start run that used each option of the parameter, by plotting the best versus worst obtained upper bounds in a scatter plot for all instances and box plots for each group of instances.

The box plot in particular is highly valuable to understand how well each value of random factor and each strategy for next tool and next job influences the final result of a multi-start run. Here is how we construct it: first, we select one group of instances within the Catanzaro's dataset, say datB. Inside datB we have four subgroups, datB1, datB2, datB3, datB4. Instances inside one specific subgroup have very similar characteristics, as they share the same number of jobs, the same number of tools and the same magazine capacity. Then we take of one the three variables we are analyzing: random factor, next tool strategy and next job strategy. Let's say we take next tool strategy in this case, for which we have two possibilities: Most Frequent (MF) and Most Joint-Frequent (MJF). Then, for each combination pair of subgroup and next tool strategy we will calculate the column of the box plot. For example, when creating the B1 - MF column, we take the obtained upper bound for every multi-start iteration for B1 instances for which we have used Most Frequent as the next tool strategy of the iteration. We then get the minimum and maximum values and the 25th, 50th and 75th percentiles and plot the column. Notice that for each pair subgroup and next tool strategy we will have 60 values to build the box plot: there are 10 instances per subgroup and 6 out of the 12 multi-start iterations

use each one of the next tool strategies. Those 60 values are the ones we use to build this column.

The main idea of the box plots is to take one specific subgroup and compare the obtained bounds by different values of random factor, and different strategies of next tool and next job, between their options. It allows us to understand not only which value or strategy got the single best result for a subgroup but also how much it tends to find different solutions and if it can lead to bad solutions sometimes. This way we might provide valuable insight to future work that can start from the values and strategies that worked best and further improve the quality of the solutions.

4.3.1 Random Factor results

Figure 4.4 shows the scatter plot including all instances and comparing the best and worst obtained upper bounds values before applying the KTNS policy for each instance for each value of Random Factor (0.1 and 0.3). We can observe that there is not much visible difference between the two values, neither in the best nor in the worst obtained values. In general, they score very closely to one another for any fixed instance.

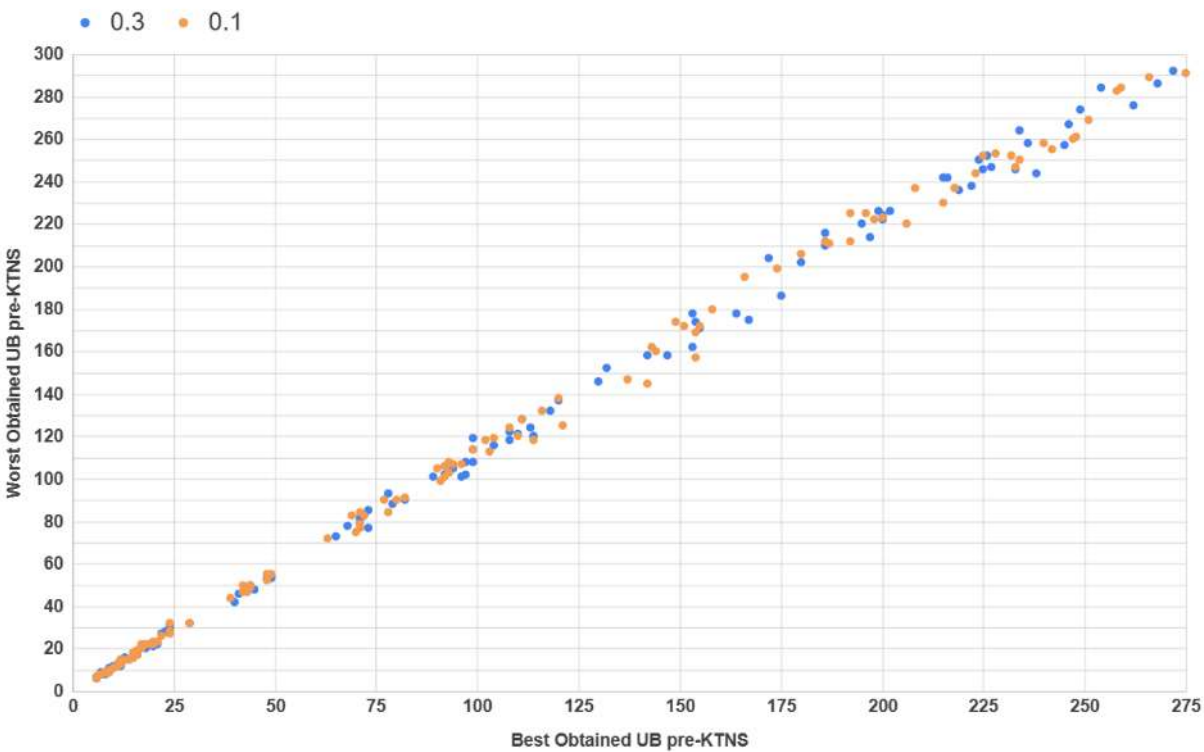


Figure 4.4: Distribution of instances for best obtained upper bound before applying KTNS and worst obtained upper bound before applying KTNS for each multi-start run, both in number of switches, separating for Random Factor used in the run.

Figure 4.5 shows the box plot considering the upper bound values obtained before applying the KTNS policy for each sub-group of instances of group datB for each value of Random Factor (0.1 and 0.3). The value of 0.1 obtained the best minimum values and also the best maximum values for all of the four subgroups, indicating a slightly better performance than the runs with 0.3. Only for the subgroup B4 did the 0.3 value obtain a better 25th percentile, but still struggled to get the best results in other parts of the distribution.

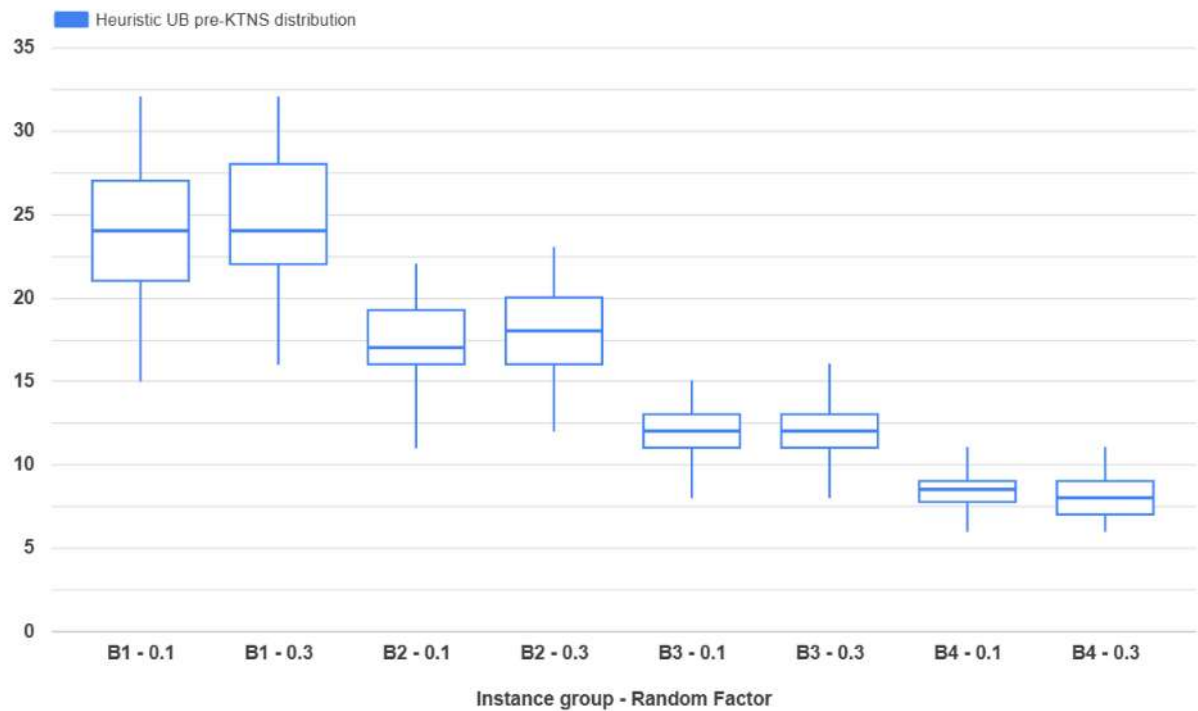


Figure 4.5: Box plot of the obtained upper bound values before applying the KTNS policy for each subgroup of instances of datB and each value of Random Factor.

Figure 4.6 shows the box plot considering the upper bound values obtained before applying the KTNS policy for each sub-group of instances of group datC for each value of Random Factor (0.1 and 0.3). Once more the value of 0.1 obtained better results, however for this group the values were even closer. 0.1 obtained the best minimum values for 3 subgroups, but the 25th percentile was tied for also 3 of the subgroups. The worst obtained upper bounds for each value were also similar, with a slight advantage for the 0.3 value.

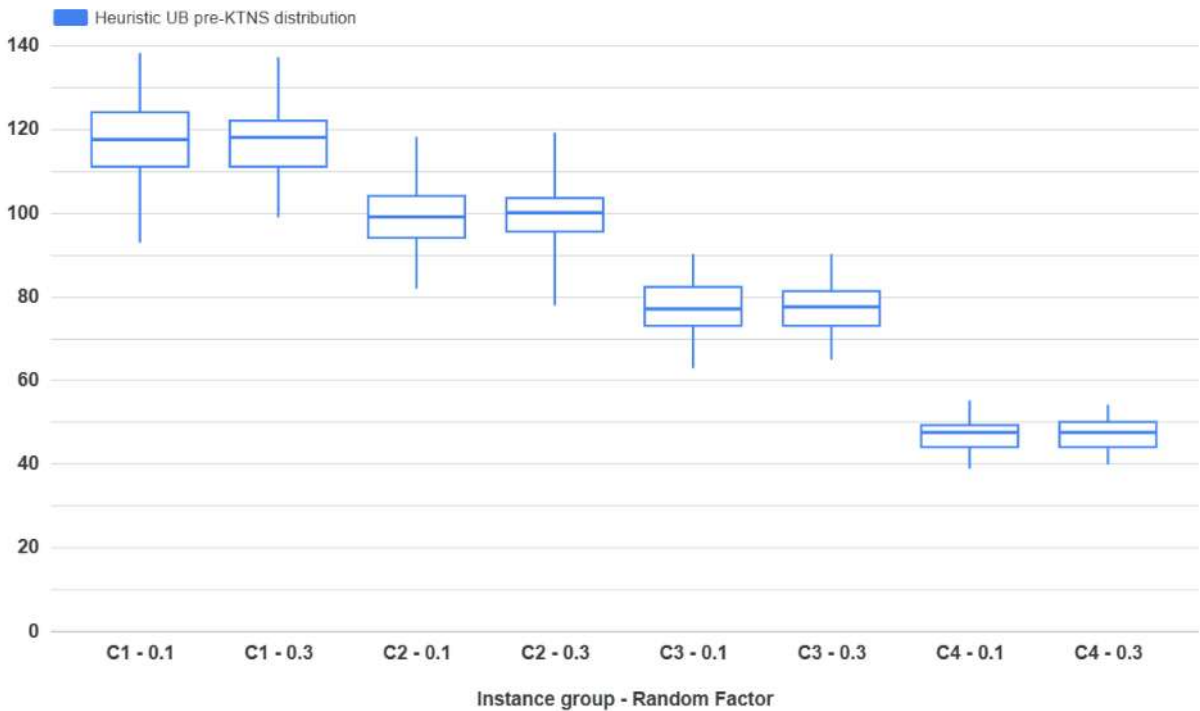


Figure 4.6: Box plot of the obtained upper bound values before applying the KTNS policy for each subgroup of instances of datC and each value of Random Factor.

Figure 4.7 shows the box plot considering the upper bound values obtained before applying the KTNS policy for each sub-group of instances of group datD for each value of Random Factor (0.1 and 0.3). Once again the results were not very different between the values, but we can see that the value of 0.1 resulted in slightly better results: better minimum bounds and better maximum bounds for 3 out of 4 subgroups (D1, D2, D3). The median, however, was very similar between values for all subgroups.

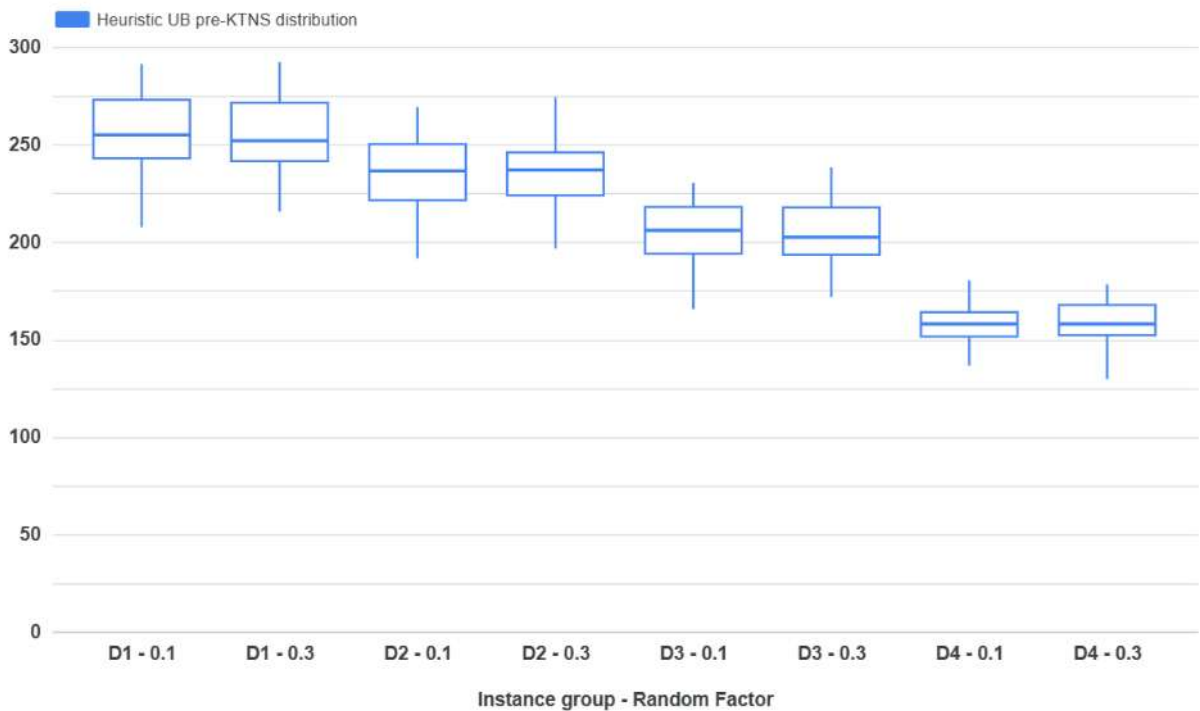


Figure 4.7: Box plot of the obtained upper bound values before applying the KTNS policy for each subgroup of instances of datD and each value of Random Factor.

In general, we may conclude that the value of random factor was not a crucial definition, at least between the two tested values. Medians between the two values were almost identical for almost all of the 12 studied subgroups. The best bound found for each subgroup, however, was frequently better for 0.1 value, which could indicate that less randomness could lead to better results. This makes sense, as the less randomness, the more we apply the logic determined by each of the strategies for next tool and next job, using some rational.

4.3.2 Next Tool Strategies results

Figure 4.8 shows the scatter plot including all instances and comparing the best and worst obtained upper bounds values before applying the KTNS policy for each instance for each option of Next Tool Strategy: Most Frequent (MF) and Most Joint Frequent (MJF). We can observe again that for both strategies have very similar general behaviour, and no much difference is clear between them in any of the two metrics (worst upper bound and best upper bound) for each instance.

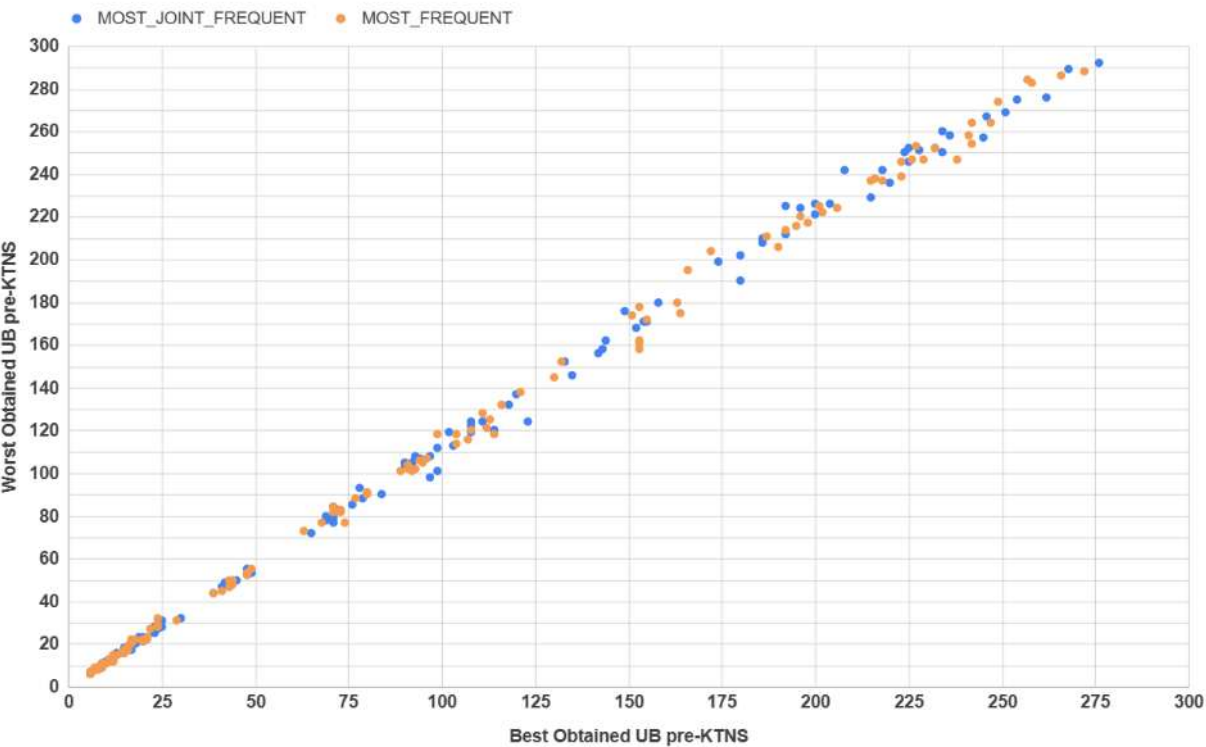


Figure 4.8: Distribution of instances for best obtained upper bound before applying KTNS and worst obtained upper bound before applying KTNS for each multi-start run, both in number of switches, separating for Next Tool Strategy used in the run.

Figure 4.9 shows the box plot considering the upper bound values obtained before applying the KTNS policy for each sub-group of instances of group datB for each option of Next Tool Strategy: Most Frequent (MF) and Most Joint Frequent (MJF). In the graph we can observe that the Most Joint-Frequent strategy had a wider range of solution values, having most of the best results and also the worst results for many subgroups of instances, indicating that it varies more than the Most Frequent strategy. The median, however, was lower when using Most Frequent for all four subgroups, indicating a slightly better performance for this strategy in this group datB.

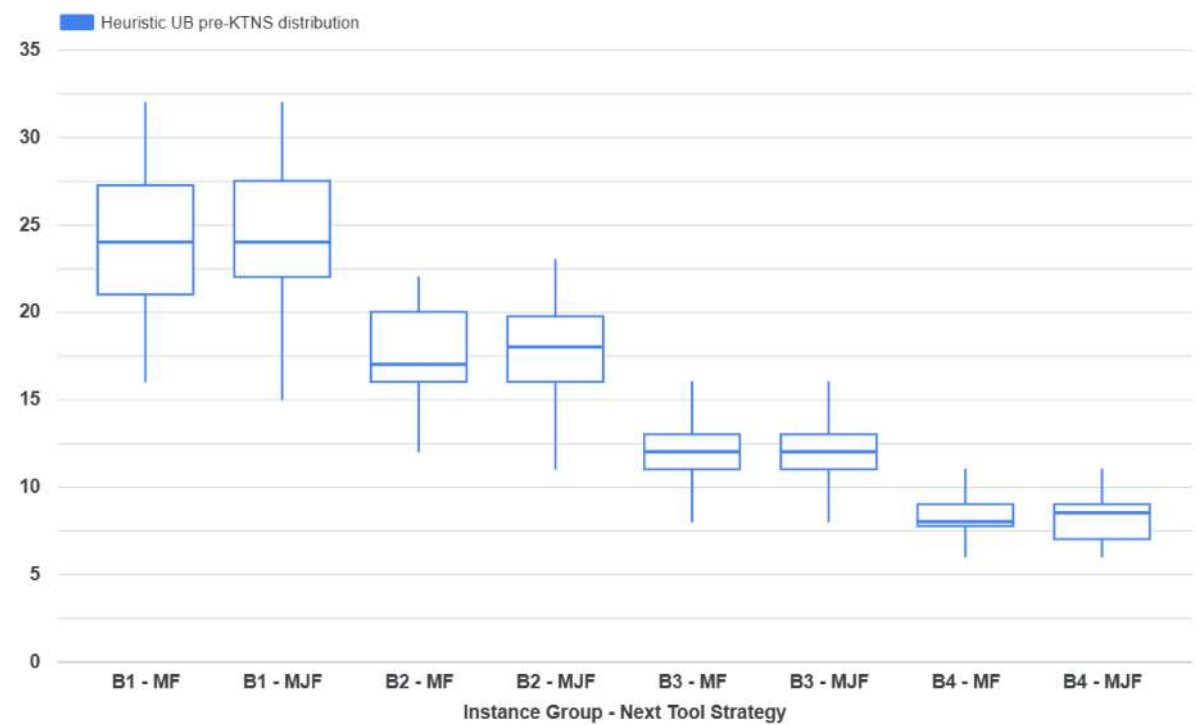


Figure 4.9: Box plot of the obtained upper bound values before applying the KTNS policy for each subgroup of instances of datB and each option of Next Tool Strategy.

Figure 4.10 shows the box plot considering the upper bound values obtained before applying the KTNS policy for each sub-group of instances of group datC for each option of Next Tool Strategy: Most Frequent (MF) and Most Joint Frequent (MJF). The results for this group of instances are much closer between both strategies: medians are almost tied in every subgroup, and each strategy found the best value in 2 out of 4 subgroups. This might also indicate that there is not enough difference between the two strategies and that it might be a good idea to try to create a third strategy capable of creating different configurations from the same base job and hence generate a different pool of configurations.

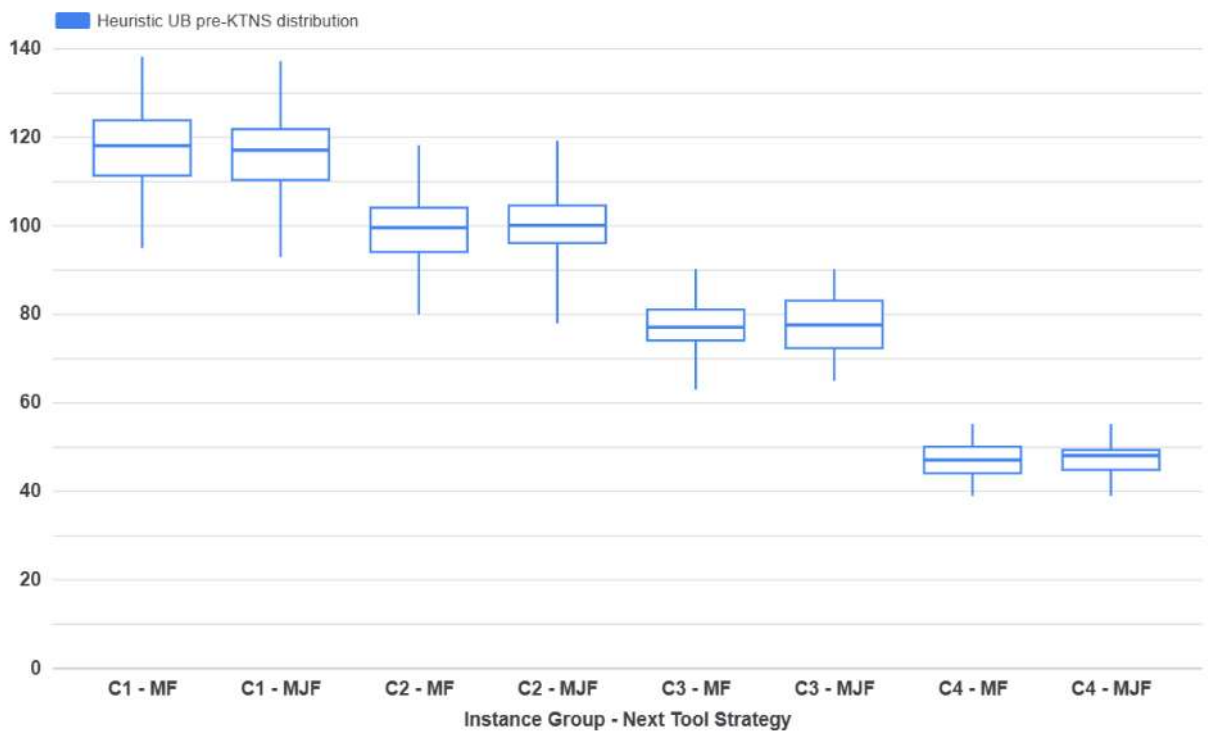


Figure 4.10: Box plot of the obtained upper bound values before applying the KTNS policy for each subgroup of instances of datC and each option of Next Tool Strategy.

Figure 4.11 shows the box plot considering the upper bound values obtained before applying the KTNS policy for each sub-group of instances of group datD for each option of Next Tool Strategy: Most Frequent (MF) and Most Joint Frequent (MJF). This graph shows again the balance between strategies, where no one is clearly better than the other. Medians are close, with a small advantage for the Most Joint-Frequent strategy. The best 25th percentile was also split, as the strategies got the best results for 2 instances each.

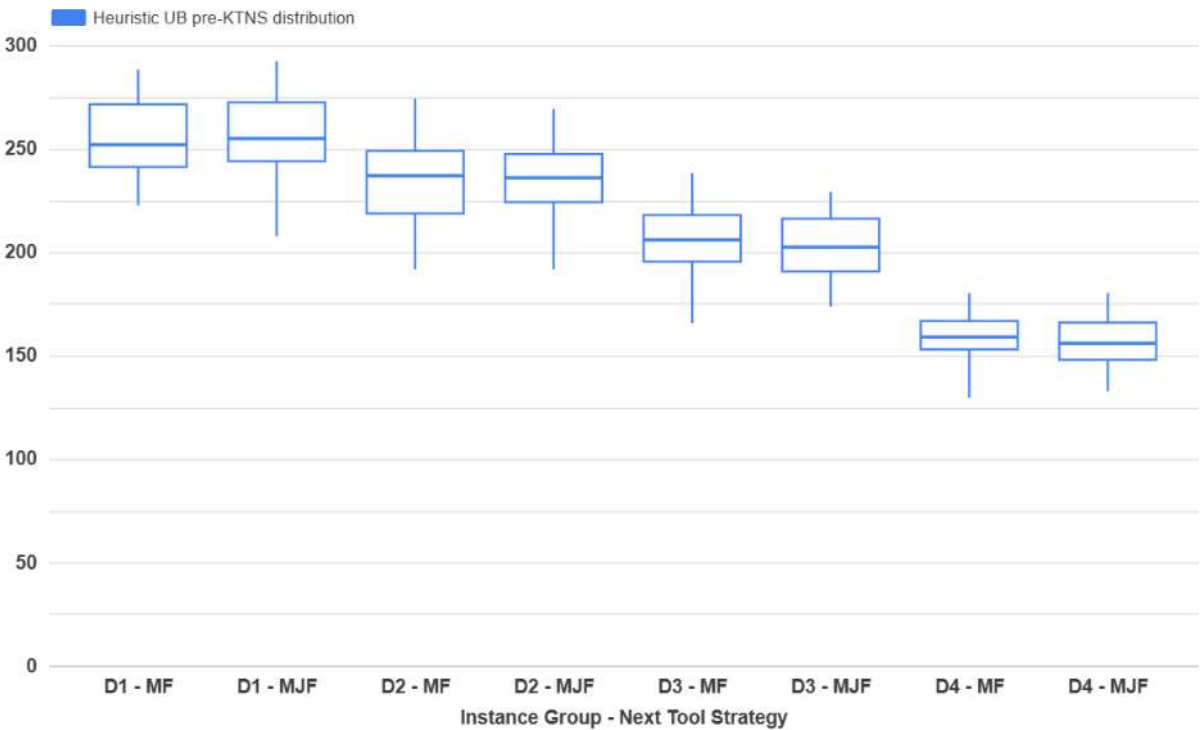


Figure 4.11: Box plot of the obtained upper bound values before applying the KTNS policy for each subgroup of instances of datD and each option of Next Tool Strategy.

We may observe from the graphs that for the Next Tool Strategy we also do not have a clear dominance from one strategy to the other: in all datB, datC and datD instances, best results were similar for many cases and have shown advantage for both strategies in different subgroups and in different parts of the distribution. It seems like the median value of Most Frequent is generally equal or lower to Most Join Frequent, but results were fairly similar and there was no clear winner in terms of general performance between those strategies. As stated in previous analysis, this might indicate a gap for future works to explore: new next tool strategies with distinct rationals when comparing to MF and MJF that are based on somewhat similar ideas.

4.3.3 Next Job Strategies results

Figure 4.12 shows the scatter plot including all instances and comparing the best and worst obtained upper bounds values before applying the KTNS policy for each instance for each option of Next Job Strategy: Least Covered (LC), Most Available Configurations (MAC) and Lowest Configuration Share (LCS). In this case, we can easily see that for the same clusters of instances, the results from the Least Covered strategy were the best,

4.3. Comparison between multi-start strategies

79

in both metrics worst upper bound and best upper bound. The other two strategies do not show much difference between them, except for some instances for which the Lowest Config Share strategy showed a slightly higher worst obtained upper bound in comparison with other points in the graph with same best obtained upper bound, which could indicate that some iterations diverge from the usual configurations and generate some bad configuration pools.

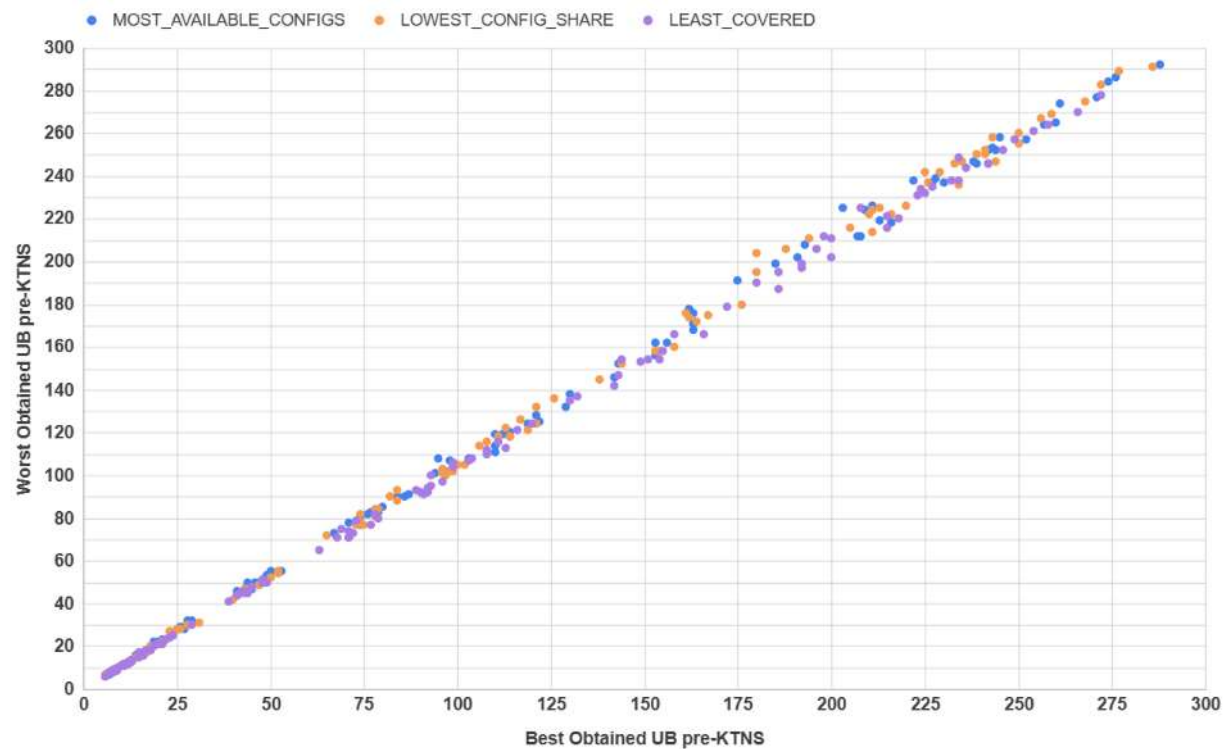


Figure 4.12: Distribution of instances for best obtained upper bound before applying KTNS and worst obtained upper bound before applying KTNS for each multi-start run, both in number of switches, separating for Next Job Strategy used in the run.

Figure 4.13 shows the box plot considering the upper bound values obtained before applying the KTNS policy for each sub-group of instances of group datB for each option of Next Job Strategy: Least Covered (LC), Most Available Configurations (MAC) and Lowest Configuration Share (LCS). In this graph we confirm our previous conclusions in regards to having a clear winner in terms of performance: the Least Covered strategy to define the next base job had by far the best results when comparing to the other two strategies: it outperformed the other strategies in every part of the distribution (minimum, 25th, 50th and 75th percentiles and maximum values) for every one of the 4 subgroups in datB. The Lowest Configuration Share strategy had intermediate results, having overall better results than the Most Available Configurations strategy in almost all cases, specially for the median, where it was tied or better than MAC strategy in all 4 subgroups.

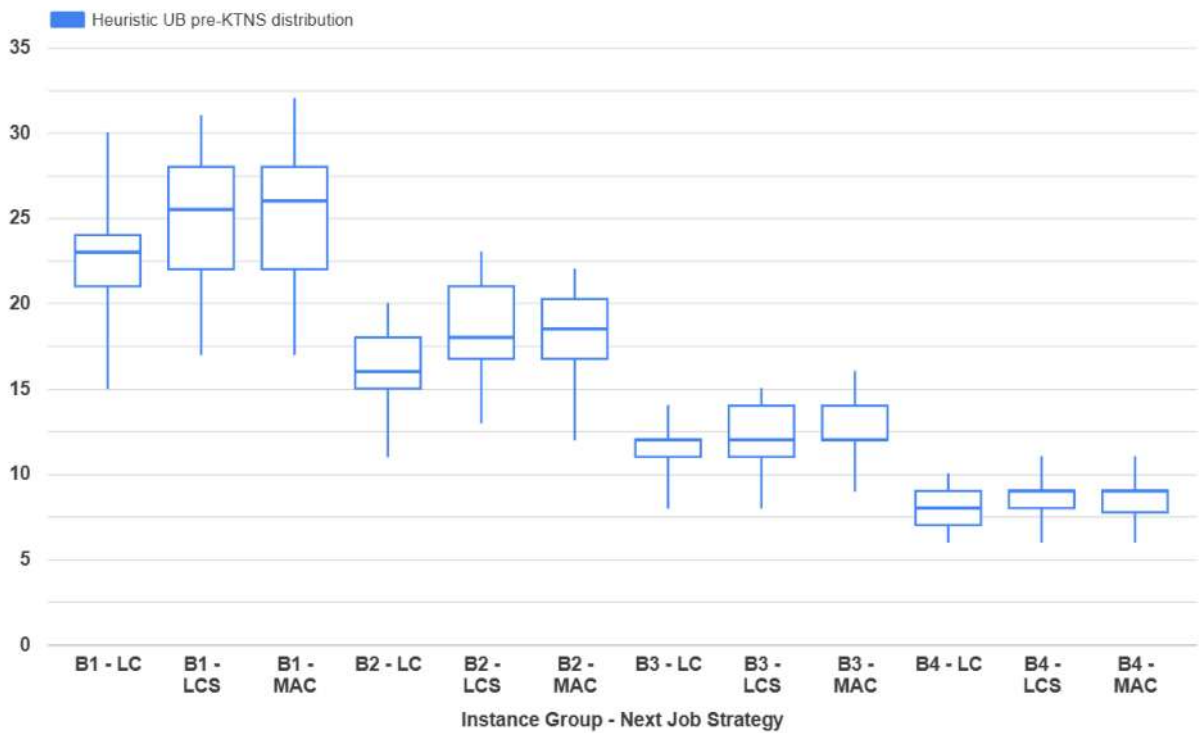


Figure 4.13: Box plot of the obtained upper bound values before applying the KTNS policy for each subgroup of instances of datB and each option of Next Job Strategy.

Figure 4.14 shows the box plot considering the upper bound values obtained before applying the KTNS policy for each sub-group of instances of group datC for each option of Next Job Strategy: Least Covered (LC), Most Available Configurations (MAC) and Lowest Configuration Share (LCS). Those instances also show the same trend seen in previous results, where Least Covered strategy had the best results for every metric for every subgroup. In this case, the results were even more impressive, as for 3 out of 4 subgroups this strategy resulted in a 75th percentile that was tied or better than the 25th percentile of both other strategies. In regards to the other two strategies, this time the results were much closer, with almost identical medians for all cases. However, there is a small advantage again for the Lowest Configuration Share strategy, as it has beaten the Most Available Configurations in the minimum value for 3 out of 4 instances and in the maximum value for 2 out of 4 instances, with 2 ties.

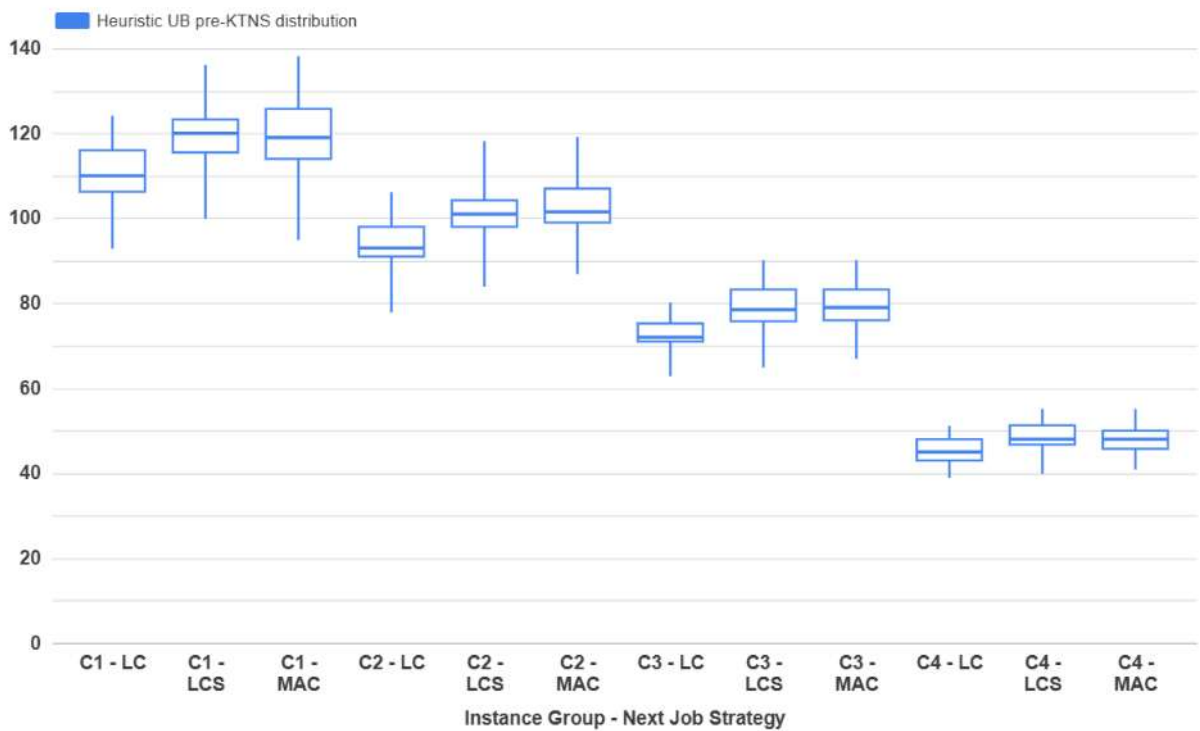


Figure 4.14: Box plot of the obtained upper bound values before applying the KTNS policy for each subgroup of instances of datC and each option of Next Job Strategy.

Figure 4.15 shows the box plot considering the upper bound values obtained before applying the KTNS policy for each sub-group of instances of group datD for each option of Next Job Strategy: Least Covered (LC), Most Available Configurations (MAC) and Lowest Configuration Share (LCS). The graph confirms one more time that the Least Covered strategy outperformed the other two strategies, again obtaining the best results in all 5 parts of the distribution for all 4 subgroups in datD. This indicates that this dominance in regards to the other strategies is not conditional to the instance sizes, instead it is actually more efficient for creating configurations regardless of the instance in the dataset from Catanzaro. Finally, the results between the other two strategies were again very similar. The 25th percentile is almost identical between them for every subgroup in datD, and the medians follow the same pattern.

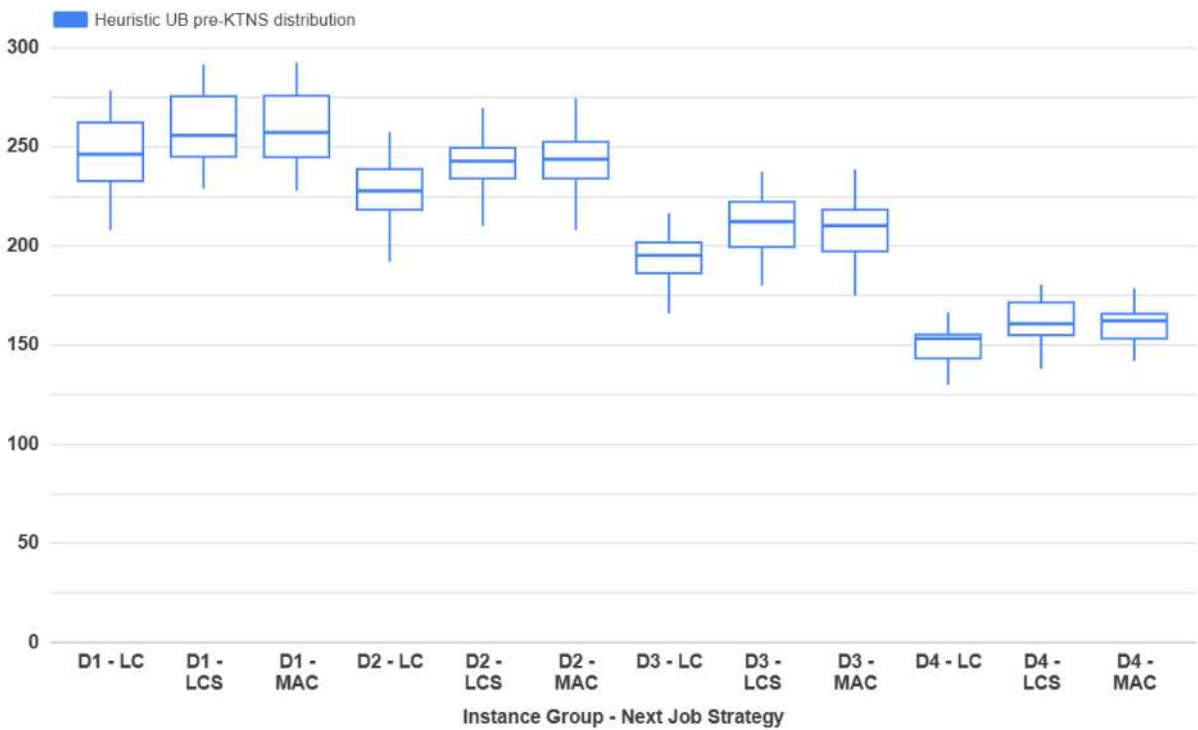


Figure 4.15: Box plot of the obtained upper bound values before applying the KTNS policy for each subgroup of instances of datD and each option of Next Job Strategy.

The results from this section clearly show that the Least Covered strategy was the most successful of the three proposed Next Job Strategies and thus deserves further work and maybe variations.

For future work, one might employ variations of this strategy by trying to tackle its main issues while being able to keep its efficiency that derives from starting the configurations by many different jobs and not getting stuck in one job in particular. We can also conclude that this is a very important strength for a Next Job Strategy as the Lowest Configuration Share strategy also performed better than the Most Available Configurations strategy. The LCS was develop to be a middle ground between the LC and the MAC, and the results show that it was really that.

Comparing this results with the results for random factor and Next Tool Strategy options, this also points to a higher influence of defining the next job to be used as base for a new configuration than the influence of defining the next tool in a configuration of even how often we select the next tool randomly. It makes sense, as for many instances we have jobs that occupy a considerable part of the magazine capacity and thus define a large portion of the new configuration being created.

Chapter 5

Conclusions

To do.

Chapter 6

Future Work

6.1 An idea for a future work: a Clustering method

The core reason why creating configuration generation heuristics is necessary is because of the factorial nature of the configurations. If it was computationally feasible, we would simply enumerate all of the configurations for which at least one job $j \in J$ is satisfied and solve the formulation with those configurations, obtaining optimal results. That enumeration is usually not possible for the whole problem, but it might be possible - or at least in part - for a smaller part of the same problem. Hence, another possibility for tackling the large universe of magazine configurations is to try to cluster jobs according to a dissimilarity metric and then apply a greedy generation of the configurations inside this cluster. Finally, it might be important to try to link every pair of cluster available to provide smoother transitions between them. This is thus a three-step heuristic method:

6.1.1 Clustering phase

In the Clustering phase, we aim to group jobs in smaller clusters based on how similar are the tool sets that two jobs require.

Some parameters can heavily impact the results obtained by this phase:

- **Clustering method:** the two methods that are more suitable for this kind of clustering are k-means and k-medoids. While k-means is more efficient for large datasets, k-medoids is more flexible in terms of distance metrics.
- **Number of clusters k :** considering that the clusters are groups for which we expect little to none tool switches, we might aim for a small number of clusters. However, note that, if all configurations of a cluster were enumerated, then the case with $k = 1$ is the complete problem, i.e. enumerating every configuration that

satisfies at least one job. We then face a tradeoff situation where fewer clusters are more computationally costly but can result in fewer tool switches and more clusters are computationally cheaper but can result in more tool switches. For the definition of k , there are many methods such as the elbow method and the silhouette method.

- **Dissimilarity metric or distance metric:** the metric used for defining how far apart are two jobs or tools should be adequate to the nature of the problem.

One example of clustering phase would be composed then by the k-medoids as the clustering method, $k = \lceil \frac{|J|}{b} \rceil$ and minimum tool switch as the dissimilarity metric. For this work, we propose a comparative approach considering multiple permutations of those parameters, aiming to find the best configuration for the available instances.

6.1.2 Exploitation phase

Once a cluster of jobs is defined, we face the challenge of the first defined goal: maximizing the amount of configurations that satisfy the tool requirements for two or more jobs at the same time.

Finding a configuration that satisfies more than one job is the same as finding subgroups of jobs for which the size of the union of the required tool sets for every job in the subgroup is less than or equal to the magazine capacity, or $|\bigcup_{i \in S} T_i| \leq b, S \subseteq J$.

The most trivial idea to achieve it would be to start by selecting a job of the cluster as the base. We then try to find another job in the same cluster for which the Minimum tool switch to the pair is equal to zero, meaning that there is at least one configuration that can satisfy both jobs. While the union of the tools required by the jobs satisfied by the configuration is less than the magazine capacity, it means that there is still a possibility of adding tools to the configuration, so we can go through the remaining jobs of the cluster and check if a new job can be added without violating the magazine capacity. If no remaining job can be added and there is still at least one vacant spot in the configuration, we fill the spots with the tools with greater frequency.

6.1.3 Linking phase

Once the jobs inside each cluster are connected in configurations by the exploitation phase, we then face the challenge of the second goal: find transition configurations that will help to move from one cluster to another, requiring minimal tool change in order to do it. Here we explain two possible approaches to achieve it.

The first approach is job-based. We first take every combination of cluster pair possible. For each cluster pair, our first goal is to find a pair of jobs, each job representing a cluster, for which there exists a configuration that satisfies both jobs. Naturally, as they were clustered separately, sometimes such a configuration will not exist. Nevertheless, when trying to minimize tool switches between clusters, we can simply take the minimum value of a dissimilarity metric between the two jobs, one from each cluster.

The second approach is configuration-based. We could take the configuration we already have and mix them using, for example, a crossover operator used in genetic algorithms to create new configurations that mix the features of two different configurations, one from each cluster. Another option for a configuration-based approach would be to simply take the most frequent tools inside both clusters and take the intersection between them. This kind of approach might be simpler, but has a relevant down side of requiring constant checking to ensure we do not create any configuration that does not satisfy any job.

6.2 Similarity Metrics Between Jobs

We may use different similarity metrics in order to build the clustering phase. Here we point some ideas for it.

When facing the decision of how to build configurations starting from every job $j \in J : 1 \leq |T_j| \leq b - 1$, we can define some concepts and measures of similarity and dissimilarity that might be useful for taking decisions inside the heuristics logic. When either clustering jobs or deciding neighbor jobs for straight configuration creation, we need similarity metrics for the jobs. Here we explore some possibilities of similarity metrics:

- **Jaccard similarity:** defined as $\frac{|T_i \cap T_j|}{|T_i \cup T_j|}$, normally goes from 0 to 1 but it should never be equal to 1 for our problem as two jobs can neither require exactly the same

tools nor have only 1 different tool, because in this case the tool set of one job would dominate the other and this case is already treated. It is a similar idea to the first dissimilarity metric we will discuss, but it differs in the sense that the size of the tool sets have an impact in the final score.

As the described clustering methods might be eventually added to this heuristic method (which could be transformed into a framework), here we will also describe some dissimilarity metrics that can be used in those methods, being analogous to the distance between geographical points. All dissimilarity metrics consider two jobs i, j and return a value that represents how dissimilar they are, where the more dissimilar the higher this value.

- **Unique tools:** The simplest idea would be to consider how many tools one of two jobs requires and the other does not: $|T_i \cup T_j| - |T_i \cap T_j|$. The more tools required by only one of the jobs, the more dissimilar they are. This metric would always be in the range $[2, 2 \times b]$. The minimum value would be 2 for the reasons mentioned above. The most dissimilar two jobs could be is the case where both tool sets are of size b and no tool is shared between them.
- **Minimum tool switch:** Note that the metric above is not sensitive to the magazine capacity. This can generate the following situation: considering an SSP instance with $b = 4$, two jobs $i, j : T_i = \{1\}, T_j = \{2\}$. Another two jobs $k, l : T_k = \{1, 2, 3, 4\}, T_l = \{1, 2, 3, 5\}$. For the first metric, both pairs of jobs i, j and k, l have a minimum dissimilarity of 2. Also, for the Jaccard similarity metric, i, j have a lower similarity score than k, l . However, i and j may be sequenced consecutively and not require any tool switch, whilst k and l may not. Hence we may want to enhance the metric above by adding a capacity component and consider instead the best case scenario for the number of tool switches between two consecutive jobs, given by $|T_i \cup T_j| - b$. However, note that this metric could result in a negative value, so we may prefer two of its version capping it in zero and considering that all pairs of jobs for which a tool switch is not mandatory have the same distance between them: $\max(0, |T_i \cup T_j| - b)$;
- **Job pair requirements:** We could also consider $|T_i \cup T_j|$, which is simply the total amount of different tools required to process both jobs. It is also not capacity sensitive, but it favors pairs of jobs that require smaller tool sets, which might be desirable when considering that two jobs are close neighbors. It also prevents the frequent ties that might be generated with the many different forms of having a distance of 0 in the minimum tool switch metric.

References

- [1] M.A. Al-Fawzan and K.S. Al-Sultan. A tabu search based algorithm for minimizing the number of tool switches on a flexible machine. *Computers Industrial Engineering*, 44(1):35–47, 2003.
- [2] Jhon Edgar Amaya, Carlos Cotta, and Antonio J. Fernández-Leiva. Solving the tool switching problem with memetic algorithms. *AI EDAM*, 26(2):221–235, 2012.
- [3] Dorothea Calmels. The job sequencing and tool switching problem: state-of-the-art literature review, classification, and trends. *International Journal of Production Research*, 57(15-16):5005–5025, 2019.
- [4] Daniele Catanzaro, Luis Gouveia, and Martine Labbé. Improved integer linear programming formulations for the job sequencing and tool switching problem. *European Journal of Operational Research*, 244(3):766–777, 2015.
- [5] A.A. Chaves, L.A.N. Lorena, E.L.F. Senne, and M.G.C. Resende. Hybrid method with cs and brkga applied to the minimization of tool switches problem. *Computers Operations Research*, 67:174–183, 2016.
- [6] Antônio Augusto Chaves, Edson Luiz França Senne, and Horacio Hideki Yanasse. Uma nova heurística para o problema de minimização de trocas de ferramentas. *Gestão Produção*, 19(1):17–30, 2012.
- [7] Rafael Colares, Mauricio Cardoso de Souza, and Annegret Wagler. Personal communication. 2024.
- [8] Yves Crama. Combinatorial optimization models for production scheduling in automated manufacturing systems. *European Journal of Operational Research*, 99(1):136–153, 1997.
- [9] Tiago Tiburcio da Silva, Antônio Augusto Chaves, and Horacio Hideki Yanasse. A new multicommodity flow model for the job sequencing and tool switching problem. *International Journal of Production Research*, 59:3617–3632, 2021.
- [10] G. Dantzig, R. Fulkerson, and S. Johnson. Solution of a large-scale traveling-salesman problem. *Journal of the Operations Research Society of America*, 2(4):393–410, 1954.

- [11] Alain Hertz, Gilbert Laporte, Michel Mittaz, and Kathryn E. Stecke. Heuristics for minimizing tool switches when scheduling part types on a flexible machine. *IIE Transactions (Institute of Industrial Engineers)*, 30(8):689 – 694, 1998.
- [12] Gilbert Laporte, Juan José Salazar-González, and Frédéric Semet. Exact algorithms for the job sequencing and tool switching problem. *IIE Transactions*, 36(1):37–45, 2004.
- [13] Lyle McGeoch and Daniel Sleator. A strongly competitive randomized paging algorithm. *Algorithmica*, 6:816–825, 1991.
- [14] Jordana Mecler, Anand Subramanian, and Thibaut Vidal. A simple and effective hybrid genetic search for the job sequencing and tool switching problem. *Computers Operations Research*, 127:105153, 2021.
- [15] Gustavo Silva Paiva and Marco Antonio M. Carvalho. Improved heuristic algorithms for the job sequencing and tool switching problem. *Computers Operations Research*, 88:208–219, 2017.
- [16] Petrică C. Pop, Ovidiu Cosma, Cosmin Sabo, and Corina Pop Sitar. A comprehensive survey on the generalized traveling salesman problem. *European Journal of Operational Research*, 314(3):819–835, 2024.
- [17] Kari Salonen, Csaba Raduly-Baka, and Olli S. Nevalainen. A note on the tool switching problem of a flexible machine. *Computers Industrial Engineering*, 50(4):458–465, 2006. Sustainability and Globalization: Selected papers from The 32nd. ICCIE.
- [18] Stefan Schwerdfeger and Nils Boysen. Order picking along a crane-supplied pick face: The sku switching problem. *European Journal of Operational Research*, 260(2):534–545, 2017.
- [19] Christopher S. Tang and Eric V. Denardo. Models arising from a flexible manufacturing machine, part i: Minimization of the number of tool switches. *Operations Research*, 36(5):767–777, 1988.
- [20] Nguyen Van Hop and Nagendra N. Nagarur. The scheduling problem of pcbs for multiple non-identical parallel machines. *European Journal of Operational Research*, 158(3):577–594, 2004.
- [21] Horacio Hideki Yanasse, Rita de Cássia Meneses Rodrigues, and Edson Luiz França Senne. Um algoritmo enumerativo baseado em ordenamento parcial para resolução do problema de minimização de trocas de ferramentas. *Gestão Produção*, 16(3):370–381, Jul 2009.